8주차 · 실습 워크북

AI 에이전트 클라우드 실습 워크북 — minikube 위에 온프레미스 에이전트 올리기

클라우드의 층 — 엣지 · 로드 밸런서 · 오토스케일링 · 게이트웨이 · GPU 모델 서빙 · DB · 관측 · CI/CD · IaC — 을 노트북 한 대의 minikube 위에 하나씩 설정하고, 망가뜨려 보고, 재면서 익힌다.

환경	Windows 11 · WSL2 Ubuntu · Docker · minikube(docker 드라이버) · RTX 5070 Laptop 8GB
장마다	완료 목표 → 그림 → 실행(명령 · 행별 해설) → 관찰 → 장애·복구 → 정리 → 성공 확인
익히기	실제 클라우드(AWS)에서는 · 연습(바꿔 보고 관찰) · 이해 확인 문제 · 면접 질문 — 정답은 부록
원본	docs/클라우드_실습_워크북.md · 측정 원본 reports/cloud_w8.md
판	2026-10-12 · 커밋 f042718 · github.com/moongu0891-ship-it/onprem-agent

13장
클러스터 하나에 층마다 한 장

58초
GPU 모델 서버 두 번째 시작 (첫 시작 2시간 40분)

409 → 200
Pod 를 죽여도 승인이 이어지게

1분 30초
make cloud-up 다시 올리기

7주차까지 만든 에이전트(상담 화면 · 에이전트 그래프 · 업무 기능(도구) 서버 · 모델 서버 · 게이트웨이)를 **실제 클라우드 서비스의 층**대로 minikube 위에 올린다. 형식은 'Minikube에서 AWS EKS까지' 워크북과 같다 — 장마다 **완료 목표 → 그림 → 실행(명령 한 줄씩 해설) → 관찰 → 장애·복구 → 정리 → 성공 확인**. 그리고 이 저장소답게 장마다 **잰다**.

- 환경: Windows 11 · WSL2 Ubuntu · Docker · minikube(docker 드라이버) · RTX 5070 Laptop 8GB · WSL2 메모리 15GB
- 상용 클라우드 없이 노트북 한 대로. 진짜로 못 만드는 칸(전 세계 CDN, 노드 자동 확장)은 흉내와 설명으로 채우고 그렇게 적는다.
- 명령은 모두 `~/onprem-agent` 에서 실행한다. 결과는 `results/cloud/` 에 남고(Git 제외), 확정본은 `reports/cloud_*.md` 로 옮긴다.

차례

00 **실습 지도와 사용 방법**
어느 층을 어느 장에서 채우는지, 노트북 메모리를 어떻게 나눠 쓰는지 이해한다.

01 **GPU 를 클러스터에 넘길 수 있는가**
이 판정이 뒤 장의 모양을 정한다
minikube 안의 Pod 가 nvidia-smi 로 노트북 GPU 를 보는지 확인하고, 결과를 적는다.

02 **클러스터 둘러보기**
노드 · 네임스페이스 · 첫 Pod
노드 하나짜리 클러스터의 자원과 작업 구역을 읽고, 상담 화면 Pod 하나를 띄워 본다(이미지는 03장에서 만들므로 여기선 공개 이미지로 흐름만).

03 **에이전트 이미지 만들기**
코드를 상자 하나에
7주차의 상담 화면·에이전트 그래프·창구 서버를 담은 컨테이너 이미지 onprem-agent:v1 을 만들어 minikube 노드에 넣고, 크기와 만드는 시간을 잰다.

04 **설정과 비밀을 밖으로**
ConfigMap · Secret · 상담 화면 Pod 하나
창구 정의(desk.yaml)를 ConfigMap 으로, 모델 키를 Secret 으로 밖에 두고, 상담 화면 Pod 하나를 띄워 브라우저로 연다. 이미지를 다시 만들지 않고…

05 **나눠 주기와 무상태**
상담 화면 ×3, 로드 밸런서, Pod 를 죽여 보기
상담 화면 Pod 를 3개로 늘려 Service 가 요청을 나눠 주는 것을 세고, 승인을 기다리는 상담 중에 그 Pod 를 죽여 상태가 Pod 안에 있으면 무슨 일이 생기는…

06 **자동으로 늘리고 줄이기**
metrics-server · HPA · 부하
상담 화면 Pod 를 CPU 사용률로 1 → 4 개까지 자동으로 늘리는 HPA 를 걸고, 부하를 주어 늘어나는 과정과 부하가 끝난 뒤 줄어드는 과정을 10초 간격으로 적는다…

07 **모델 서버를 클러스터 안으로**
GPU Pod · PVC · 상담 화면을 실제 모델로
01장 판정 A 에 따라 SGLang(Qwen3-4B(FP8))을 GPU 를 요구하는 Pod 로 올리고, 가중치는 PVC 에 두며, 상담 화면이 규칙 모델 대신 이 모델 서…

08 **엣지**
https · 되돌림 · 방화벽 · CDN 흉내
상담 화면 앞에 Ingress-NGINX 를 두고 자체 서명 인증서로 https 를 켠다. http 는 https 로 되돌리고, 모델 서버는 상담 화면(과 09장의 게이트웨…

09 **관문**
로그인 · 요청 제한 · LiteLLM 게이트웨이
Ingress 에 기본 인증(Basic Auth)과 초당 요청 제한을 더하고, 모델 서버 앞에 LiteLLM 게이트웨이를 두어 키 없는 요청을 401 로 막는다.

10 **데이터**
PostgreSQL + pgvector 하나로, 무상태 되찾기
05장에서 확인한 "승인 대기가 Pod 안에 있다"를 고친다. 대화·승인 대기(LangGraph 체크포인터)와 검색 색인(pgvector)을 PostgreSQL Statef…

11 **관측**
/metrics · Prometheus · Grafana
상담 화면이 숫자(질문 수 · 처리 시간 분포 · 확인 대기 · 승인 · 오류)를 /metrics 로 내놓고, Prometheus 가 15초마다 긁어 가며, Grafana …

12 **CI/CD**
Jenkins 를 클러스터에, 노트북 GPU 를 일꾼으로
Jenkins 제어부를 helm 으로 클러스터에 올리고, 노트북을 인바운드 일꾼(laptop, 라벨 gpu) 으로 붙인다. 저장소에 올리면(5분마다 poll) 단위·모듈 시…

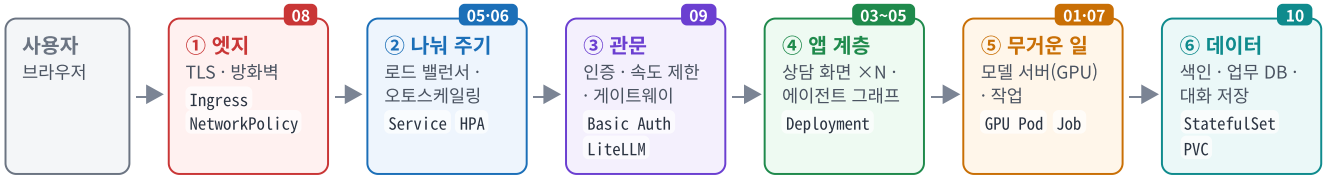
13 **IaC**
make cloud-up 한 줄, 처음부터 다시 올리는 시간
01~11장을 스크립트 하나(deploy/scripts/cloud-up.sh)로 순서대로, 다시 돌려도 되게(멱등) 묶고, 클러스터를 지운 뒤 처음부터 올리는 데 걸리는 시…

- **마치며 — 이 워크북에서 잰 것**
- **부록 — 스크립트 한 줄씩**
- **부록 — 이해 확인 정답과 해설**

00 실습 지도와 사용 방법

완료 목표 어느 층을 어느 장에서 채우는지, 노트북 메모리를 어떻게 나눠 쓰는지 이해한다.

실습 지도 **사용자에서 데이터까지 — 층마다 몇 장에서 채우나**



⑦ 옆단 — 모든 층에 걸쳐서

설정 · 비밀 ConfigMap / Secret — 04 · 관측 Prometheus · Grafana — 11 · CI/CD Jenkins · 노트북 GPU 일꾼 — 12 · IaC make cloud-up — 13 · 클러스터 자체 minikube — 02

읽는 법 왼쪽에서 오른쪽이 요청이 들어가는 길이다. 모서리 숫자가 그 층을 채우는 장. 실제 클라우드라면 ① CloudFront·WAF·ACM ② ALB·HPA·Karpenter ③ API Gateway·Cognito ④ EKS·ECS ⑤ GPU 노드 그룹·SageMaker ⑥ RDS·S3 ⑦ CloudWatch·CodePipeline·Terraform 자리다.

층 (그림의 칸)	실제 클라우드에서는	이 워크북에서 만드는 것	장	잰다
화면	브라우저가 보는 프런트엔드	7주차 상담 화면(`/`)·관리자 화면(`/admin`)을 Pod 로	03·04	이미지 크기·빌드 시간, 화면이 뜨기까지
① 엣지 · TLS·방화벽	ACM · WAF	Ingress-NGINX + 자체 서명 인증서(https), NetworkPolicy 로 모델 서버는 앱만 접근	08	http→https 되돌림, 막힌 통신 확인
① CDN	CloudFront	**없음(흉내만)** — 정적 파일에 캐시 머리말(Cache-Control) 붙이고 두 번째 요청이 304 인지 본다	08	304 비율
② 로드 밸런서	ALB · NLB	Service(ClusterIP) 가 상담 화면 Pod N 개에 나눠 준다 — 어느 Pod 가 받았는지 답 머리말로 본다	05	Pod 별 요청 수
② 오토스케일링	HPA · Auto Scaling	HPA 로 상담 화면 Pod 1 → 4 (CPU 기준) · 노드 자동 확장은 **없음(설명)**	06	부하 중 Pod 수와 p95
③ API 게이트웨이 · 인증 · 속도 제한	API Gateway · Cognito · rate limit	Ingress 에 기본 인증(Basic Auth) + 분당 요청 제한 · LiteLLM 게이트웨이가 모델 앞	09	503(거부)이 나오는 요청률
④ 앱 서버(무상태) ×N	ECS · Cloud Run	상담 화면 Pod 는 **무상태** — 대화·승인 대기는 ⑥ 의 DB 에	05	Pod 를 죽여도 승인 대기가 이어지나
④ 컨테이너 오케스트레이터	Kubernetes · ECS	minikube 자체	02	—
⑤ 모델 서빙(GPU)	SageMaker · Triton	SGLang Qwen3-4B(FP8) — **GPU 가 Pod 에 보이면 클러스터 안, 아니면 호스트 Docker 에 두고 클러스터에서 붙음**(01장이 정함)	01·07	첫 글자 시간·처리량 (3주차 측정기 그대로)
⑤ 작업 큐 · 워커	SQS · Celery	전체 시험(1시간)을 Job 으로 — 큐는 Jenkins 가 맡는다	12	—
⑤ 배치 파이프라인	Airflow · Batch	검색 색인 만들기를 Job 으로(앱 시작 전에 한 번)	10	색인 시간
⑥ 벡터 DB · 관계형 DB	pgvector · RDS	PostgreSQL + pgvector StatefulSet 하나로 둘 다(2주차 어댑터 그대로) · 대화·승인 대기 저장도 여기	10	Pod 재시작 뒤 데이터가 남나
⑥ 객체 저장소 · 캐시	S3 · Redis	PVC 에 모델·색인 파일 · 캐시는 모델 서버의 접두사 캐시(3주차) — Redis 는 **없음(설명)**	10	—
⑦ 관측	CloudWatch · Datadog	Prometheus + Grafana(가벼운 구성) + Langfuse(에이전트 추적)	11	대시보드에 p95·승인 수
⑦ CI/CD	GitHub Actions · ArgoCD	**Jenkins** 를 minikube 에, 노트북(GPU)을 일꾼으로 — 올릴 때 단위·모듈, 매일 밤 전체 시험	12	밤 시험이 돌고 빨간불 보고서가 남나

층 (그림의 칸)	실제 클라우드에서는	이 워크북에서 만드는 것	장	잰다
⑦ 설정 · 비밀	Secrets Manager	ConfigMap(창구 정의 `desk.yaml`) · Secret(게이트웨이 키, DB 비밀번호)	04	—
⑦ IaC	Terraform	`deploy/` 의 매니페스트·Helm 값 파일이 그 자체. `make cloud-up` 한 줄로 전부	13	처음부터 다시 올리는 시간

메모리 예산 (WSL2 15GB) ~~minikube 노드 6GB + 모델 서버 4~5GB~~ → **07장에서 재서 고침:** 노드 12GB(`docker update`) 안에 모델 서버 7.9GB(안정, 첫 시작 땐 11GB 넘게) + 상담 화면 0.8GB + 제어부·DB·관측 2~3GB. 모델 서버가 뜰 때는 다른 것을 내려 둔다. 전체 시험 중 임베더 2GB 는 노드 밖(호스트). **전체 시험(12장)은 관측·Jenkins 외 실습 서비스를 내린 상태에서만** 돌린다 — Jenkins 가 시작 전에 확인한다. 장마다 '이 장에 필요한 것만' 띄우고 끝에 내린다.

장 순서와 의존 00 → 01(GPU 판정) → 02 → 03 → 04 → 05 → … 순서대로. 01 의 결과가 07 의 모양을 정하므로 **01 을 먼저 노트북에서 돌리고 결과를 적은 뒤** 뒤 장을 쓴다(이 문서는 그 순서대로 자란다).

이 워크북으로 익히는 법

장마다 실습 뒤에 네 가지 절이 붙는다. 순서대로 하면 '돌려 봤다'가 '설명할 수 있다'가 된다.

절	하는 일	왜
실제 클라우드에서는	이 장에서 minikube 로 한 설정이 AWS(EKS)에선 무엇인지 대응표로	면접 · 실무는 클라우드 이름으로 묻는다
연습 — 바꿔 보고 관찰하기	값을 바꾸거나 일부러 망가뜨리기 전에 **결과를 예측해 적고**, 실행해 맞춰 본다	설정은 바꿔 봐야 무엇을 하는지 안다
이해 확인	객관식 · 단답 4문제 — 정답과 해설은 맨 뒤 부록	스스로 시험
면접에서 이렇게 묻는다	자주 나오는 질문과 30초 답 — 이 워크북에서 잰 숫자로	경험을 숫자로 말하는 연습

코드와 해설을 읽는 순서

각 코드 상자의 왼쪽 숫자는 해설을 찾는 행 번호다. 터미널에는 숫자를 빼고 명령만 넣는다. 상자 이름은 `장-번호`(예 `01-003`). 행 끝의 `\` 는 다음 줄까지 한 명령이다. 모든 명령은 `~/onprem-agent` 에서, WSL2 Ubuntu 터미널에서 실행한다. `kubectl`·`minikube`·`docker` 는 가상환경과 무관하지만, `python scripts/…` 와 `make …` 는 이 저장소의 가상환경에서 — 프롬프트가 `(.venv)` 가 아니면 `deactivate; source .venv/bin/activate`. 긴 스크립트는 `deploy/scripts/` 에 있고 부록에서 한 줄씩 읽는다.

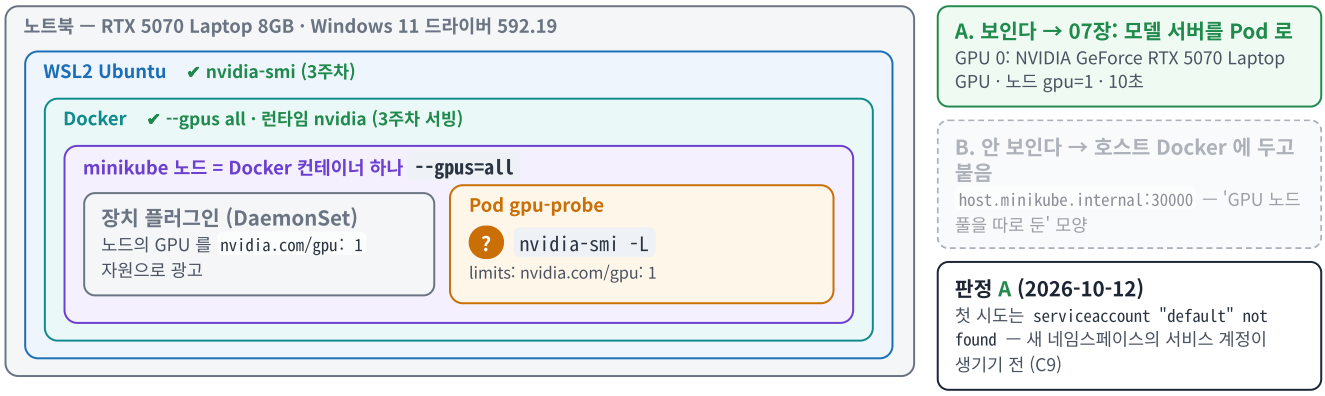
표현	읽는 방법
`-n lab`	조작할 네임스페이스(작업 구역) 이름. 이 워크북은 전부 `lab`
`-p agent`	minikube 프로필(클러스터 이름). 이 워크북은 전부 `agent`
`kubectl get/describe/logs`	조회 / 상세와 사건 / 컨테이너 출력
`--overrides='{...}'`	명령 한 줄로 Pod 를 만들 때 자원 요구 같은 세부를 JSON 으로 덧붙임
성공 판정	종료 코드 0 + 각 장의 '성공 확인' 에 적은 출력

01 GPU 를 클러스터에 넘길 수 있는가

이 판정이 뒤 장의 모양을 정한다

완료 목표 minikube 안의 Pod 가 `nvidia-smi` 로 노트북 GPU 를 보는지 확인하고, 결과를 적는다.

그림 01 GPU 가 어디까지 보이나



핵심 GPU 는 네 겹(WSL2 → Docker → 노드 컨테이너 → Pod)을 통과해야 한다. 앞 두 겹은 3주차에 확인했고, 이 장은 뒤 두 겹만 잰다.

minikube docker 드라이버는 노드 자체가 Docker 컨테이너다. 그 안에서 또 컨테이너(Pod)가 도는 구조라, GPU 가 두 겹을 통과해야 한다. minikube 는 `--gpus all` 옵션과 NVIDIA 장치 플러그인으로 이를 지원하지만 WSL2 에서는 되는 조합이 좁다. 그래서 **먼저 잰다**.

실행

Ubuntu Bash 01-001

```
bash deploy/scripts/install-minikube.sh
minikube start -p agent --driver=docker --container-runtime=docker --gpus=all --memory=6g --cpus=4
kubectl create namespace lab
minikube -p agent addons enable nvidia-device-plugin
kubectl -n kube-system get pods | grep nvidia
```

1. kubectl · minikube · helm 을 설치한다(이미 있으면 건너뜀). 스크립트 한 줄씩 읽기는 부록.
2. 클러스터를 만든다. `-p agent`: 프로필 이름. `--driver=docker`: 노드를 Docker 컨테이너로. `--gpus=all`: 노드 컨테이너에 GPU 를 넘긴다 (docker 의 `--gpus all` 과 같음). `--memory=6g --cpus=4`: 노드 상한 — 15GB 중 모델 서버 몫을 남긴다.
3. 작업 구역 `lab` 을 만든다. 이 워크북의 모든 자원은 여기에.
4. NVIDIA 장치 플러그인을 켠다 — 노드의 GPU 를 Kubernetes 가 `nvidia.com/gpu` 자원으로 세게 한다.
5. 플러그인 Pod 가 떴는지 본다. `Running` 이어야 한다.

Ubuntu Bash 01-002

```
bash deploy/scripts/probe-gpu.sh
```

1. CUDA 기본 이미지로 Pod 하나를 띄워 `nvidia-smi -L` 을 찍고, 노드가 광고하는 GPU 수와 Pod 의 사건(Events)을 `results/cloud/01_gpu_probe.txt` 에 남긴다. 스크립트 한 줄씩 읽기는 부록.

관찰 — 둘 중 하나가 나온다

결과	보이는 것	뒤 장의 모양
A. 된다	`GPU 0: NVIDIA GeForce RTX 5070 Laptop GPU (UUID: …)` , 노드 `gpu=1`	07장에서 SGLang 을 **Pod** 로 올린다. 모델 가중치는 PVC(10장)
B. 안 된다	Pod 가 `Pending` (사건: `Insufficient nvidia.com/gpu`) 또는 `nvidia-smi` 실패, 노드 `gpu=` (빈 값)	07장에서 SGLang 은 **호스트 Docker**(3주차 compose 그대로)에 두고, 클러스터의 게이트웨이가 `host.minikube.internal:30000` 로 붙는다. 실제 클라우드로 치면 'GPU 노드 풀을 따로 둔 것'과 같은 그림

B 가 나와도 실습은 모두 된다 — 모델 서버만 클러스터 밖이다. 어느 쪽이든 결과를 아래 표에 적는다.

장애·복구 `minikube start` 가 `--gpus` 에서 실패하면 옵션을 빼고 다시 만든다(`minikube delete -p agent` 뒤 `--gpus` 없이). 그 경우는 B 다.

정리 이 장은 내리지 않는다 — 02장이 이 클러스터를 쓴다. `kubectl -n lab delete pod gpu-probe` .

성공 확인 **`results/cloud/01_gpu_probe.txt` 가 있고, 아래 표를 채웠다.**

항목	결과 (2026-10-12, 노트북)
minikube 버전 / 드라이버	v1.39.0 / docker (Kubernetes v1.37.0, 노드 CPU 4 · 6GB)
`--gpus=all` 로 start 됐나	됐다. 장치 플러그인 `nvcr.io/nvidia/k8s-device-plugin:v0.20.0` 이 `Registered device plugin for 'nvidia.com/gpu'`
노드 `nvidia.com/gpu` 광고 수	**1**
Pod 의 `nvidia-smi -L`	`GPU 0: NVIDIA GeForce RTX 5070 Laptop GPU (UUID: GPU-3e4f…)` — 이미지 받기 6.5초 포함 10초
판정 (A/B)	**A — 모델 서버를 클러스터 안 Pod 로 올린다(07장)**

첫 실행에서는 Pod 가 `Forbidden: serviceaccount "default" not found` 로 거부됐다 — 막 만든 네임스페이스의 기본 서비스 계정이 몇 초 뒤에 생기는데 그 전에 Pod 를 만들었다. 스크립트가 서비스 계정과 장치 플러그인을 기다리게 고쳤다(문제해결 이력 C9). 호스트 드라이버 592.19, Docker 런타임에 `nvidia` 있음.

실제 클라우드에서는 (AWS EKS)

이 워크북 (minikube)	AWS (EKS)	다른 점
`minikube start --gpus=all`	GPU 노드 그룹(g5 · g6 · p4d) 또는 Karpenter NodePool 에 GPU 인스턴스	노드가 처음부터 GPU 드라이버가 든 **EKS 최적화 GPU AMI** 로 뜬다
`addons enable nvidia-device-plugin`	NVIDIA device plugin DaemonSet 또는 **GPU Operator**(드라이버 · 툴킷 · DCGM 까지)	운영에선 GPU 사용률을 DCGM exporter 로 잰다
노드 하나에 GPU 하나	GPU 노드에 **taint** `nvidia.com/gpu:NoSchedule` , GPU Pod 만 toleration	비싼 노드에 일반 Pod 가 앉지 않게
`limits: nvidia.com/gpu: 1`	같다	GPU 를 나눠 쓰려면 time-slicing 이나 MIG(A100 · H100)

연습 — 바꿔 보고 관찰하기

먼저 결과를 **예측해 적고** 실행한다. 끝나면 되돌린다.

1. **GPU 를 2개 요구하면?** `probe-gpu.sh` 의 `"nvidia.com/gpu":1` 을 `2` 로 바꾼 사본으로 Pod 를 띄운다. 예측 → `Pending` , Events 에 `Insufficient nvidia.com/gpu` . 확인 `kubectl -n lab describe pod gpu-probe | tail -5` .
2. **장치 플러그인을 끄면?** `minikube -p agent addons disable nvidia-device-plugin` 뒤 `kubectl get node -o jsonpath='{.items[0].status.allocatable}'` . 예측 → `nvidia.com/gpu` 가 사라진다(이미 떠 있는 GPU Pod 는 계속 돈다). 다시 `enable` .

3. **GPU 를 요구하지 않은 Pod 에서 nvidia-smi 는?** `kubectl -n lab run nogpu --rm -it --restart=Never --image=nvidia/cuda:12.6.3-base-ubuntu24.04 -- nvidia-smi -L`. 예측 → 실패하거나 GPU 가 안 보인다. GPU 는 **요구해야** 붙는다.

이해 확인

1. Pod 가 GPU 를 쓰려면 매니페스트에 무엇을 적나? (가) `requests.memory` (나) `limits: nvidia.com/gpu: 1` (다) `hostNetwork: true` (라) `privileged: true`
2. 노드가 `nvidia.com/gpu: 1` 을 '광고'하게 만드는 것은 무엇인가?
3. GPU 가 하나인 노드에 GPU 1개를 요구하는 Pod 를 두 개 띄우면 어떻게 되나?
4. 01장 첫 실행이 `Forbidden: serviceaccount "default" not found` 로 거부된 이유는?

면접에서 이렇게 묻는다

Q. 쿠버네티스는 GPU 를 어떻게 스케줄링하나요?

GPU 는 CPU · 메모리 같은 기본 자원이 아니라 **확장 자원**입니다. 노드마다 NVIDIA 장치 플러그인이 GPU 수를 `nvidia.com/gpu` 로 광고하고, Pod 가 `limits` 로 정수 개를 요구하면 스케줄러가 남은 수가 있는 노드에 둡니다. 기본은 나눠 쓰지 않아서, 하나뿐이면 두 번째 Pod 는 Pending 입니다. EKS 에선 GPU AMI 와 플러그인(또는 GPU Operator)을 쓰고, 비싼 노드라 taint 로 GPU Pod 만 오게 합니다. 저는 노트북에서 GPU 가 WSL2 → Docker → minikube 노드 컨테이너 → Pod 네 겹을 통과하는지 먼저 재고(판정 A) 모델 서버 위치를 정했습니다.

Q. GPU 비용은 어떻게 줄이나요?

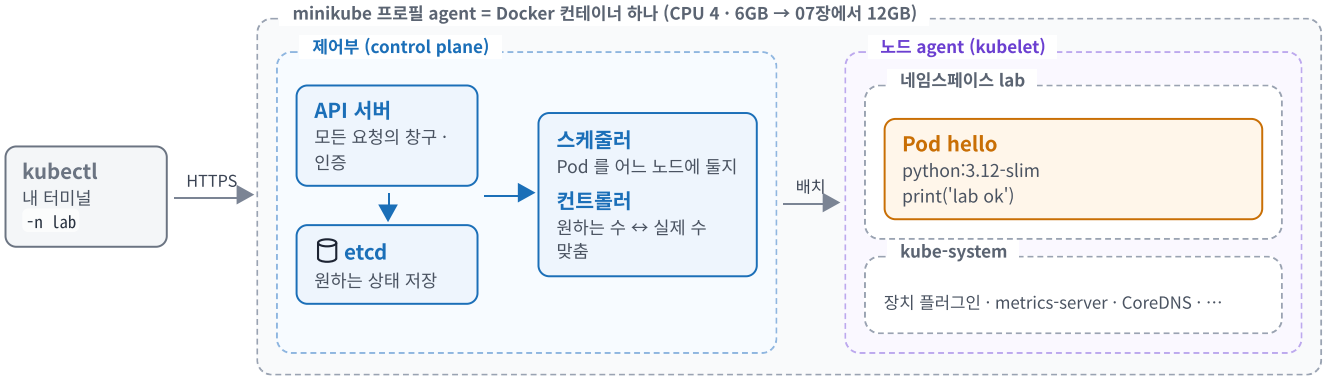
모델 서버만 GPU 노드에 두고(taint · nodeSelector), Karpenter 로 필요할 때만 노드를 띄우고, 처리량을 올립니다. 3주차에 에이전트 입력의 66~84% 가 앞 요청과 같아 접두사 캐시가 처리량을 2~3배 올리는 걸 쟀습니다 — 같은 GPU 로 더 많이 처리하는 게 첫 번째 절약입니다.

02 클러스터 둘러보기

노드 · 네임스페이스 · 첫 Pod

완료 목표 노드 하나짜리 클러스터의 자원과 작업 구역을 읽고, 상담 화면 Pod 하나를 띄워 본다(이미지는 03장에서 만들므로 여기선 공개 이미지로 흐름만).

그림 02 minikube 한 대 = 제어부 + 노드 하나



사건 순서 Scheduled → Pulling → Pulled → Created → Started — 클라우드의 모든 배포가 이 다섯 단계다. **EKS 라면** 제어부는 AWS 가 관리하고(돈 내는 부분), 노드는 EC2 노드 그룹이다.

실행

Ubuntu Bash 02-001

```
kubectl get nodes -o wide
kubectl describe node agent | sed -n '/Allocatable/,/System Info/p'
kubectl get ns
kubectl -n lab run hello --image=python:3.12-slim --restart=Never -- python -c "print('lab ok')"
kubectl -n lab wait --for=jsonpath='{.status.phase}'=Succeeded pod/hello --timeout=120s
kubectl -n lab logs hello
kubectl -n lab delete pod hello
```

1. 노드 목록과 주소·버전. `-o wide`: 내부 IP 와 컨테이너 런타임까지.
2. 노드 상세 중 '할당 가능' 자원(cpu·memory·`nvidia.com/gpu`)만 잘라 본다. 01장의 판정과 같아야 한다.
3. 네임스페이스 목록. `lab` 과 시스템용 `kube-system` 이 보인다.
4. Pod 하나를 만들어 파이썬 한 줄을 실행한다. `--restart=Never`: 끝나면 그대로 둔다(Job 처럼). `--`: 뒤는 컨테이너 안 명령.
5. Pod 가 `Succeeded` 될 때까지 기다린다(최대 120초 — 이미지 받기 포함).
6. 컨테이너 출력. `lab ok`.
7. 지운다.

관찰 `kubectl -n lab get events --sort-by=.lastTimestamp | tail` 로 Pod 하나가 만들어지는 사건 순서(Scheduled → Pulling → Pulled → Created → Started)를 본다. 클라우드의 모든 배포가 이 다섯 단계다.

장애·복구 4행에서 `ImagePullBackOff` 면 인터넷(Docker Hub) 문제다. `minikube -p agent ssh -- docker pull python:3.12-slim` 로 노드 안에서 직접 받아 본다.

정리 없음(Pod 는 7행에서 지움).

성공 확인 2행의 `Allocatable` 에 `memory` 가 6Gi 근처, 6행 출력이 `lab ok` .

실제 클라우드에서는 (AWS EKS)

이 워크북	AWS	다른 점
minikube 컨테이너 하나 = 제어부 + 노드	**EKS 제어부**(API 서버 · etcd 를 AWS 가 여러 AZ 에 운영) + EC2 노드 그룹 / Fargate	제어부는 관리형이라 시간당 요금, etcd 를 직접 만질 수 없다
`kubectl` (minikube 가 kubeconfig 를 써 줌)	`aws eks update-kubeconfig` + IAM 인증(Access Entry)	누가 무엇을 하는지 IAM ↔ RBAC 로 묶는다
네임스페이스 `lab`	팀 · 환경(dev · prod)마다 네임스페이스, ResourceQuota · LimitRange	실제론 네임스페이스마다 자원 한도를 건다

연습 — 바꿔 보고 관찰하기

1. **없는 이미지 태그를 쓰면?** `kubectl -n lab run bad --image=python:3.12-없는태그 --restart=Never` . 예측 → `ErrImagePull` → `ImagePullBackOff` . `describe` 의 Events 로 확인하고 지운다.
2. **맨 Pod 와 Deployment 의 차이.** `kubectl -n lab run solo --image=busybox -- sleep 3600` 과 `kubectl -n lab create deployment duo --image=busybox -- sleep 3600` 을 만들고 둘 다 Pod 를 지운다. 예측 → `solo` 는 사라지고 `duo` 는 새 이름으로 다시 생긴다. 이유: 컨트롤러가 원하는 수(1)를 지킨다. 끝나면 `kubectl -n lab delete deploy duo` .
3. **사건 다섯 단계 읽기.** `kubectl -n lab get events --sort-by=.lastTimestamp | tail -8` 에서 Scheduled · Pulling · Pulled · Created · Started 를 찾아 시간 차를 적는다(이미지 받기가 대부분).

이해 확인

1. 클러스터의 '원하는 상태'를 저장하는 곳은? (가) kubelet (나) etcd (다) 스케줄러 (라) kube-proxy
2. 새 Pod 를 어느 노드에 둘지 정하는 구성 요소는?
3. `kubectl run --restart=Never` 로 만든 Pod 를 지우면 다시 생기나? 왜?
4. EKS 에서 돈을 내고 AWS 에 맡기는 부분은 무엇인가?

면접에서 이렇게 묻는다

Q. 쿠버네티스 제어부는 무엇으로 이루어져 있나요?

API 서버(모든 요청의 유일한 창구 · 인증), etcd(원하는 상태를 담는 저장소), 스케줄러(Pod 를 노드에 배치), 컨트롤러 매니저(원하는 상태와 실제 상태를 계속 맞추는 반복 — Deployment 가 Pod 수를 지키는 것)입니다. 노드에는 kubelet · kube-proxy · 컨테이너 런타임이 있습니다. EKS 는 제어부를 AWS 가 여러 가용 영역에 운영해 주고, 우리는 노드와 워크로드만 관리합니다.

Q. Pod 를 직접 만들면 안 되나요?

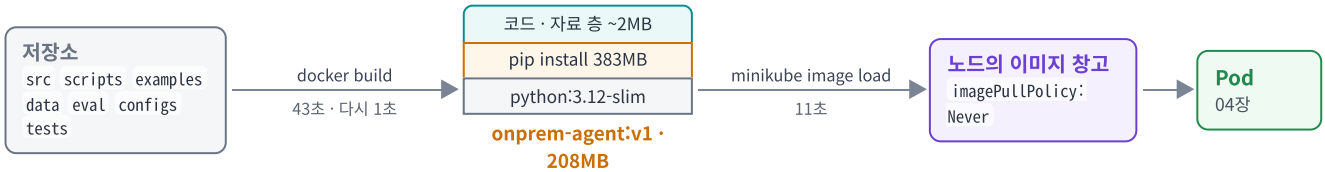
만들 수는 있지만 지켜 주는 컨트롤러가 없어서 죽으면 끝입니다. 실습에서 맨 Pod 는 지우면 사라지고, Deployment 의 Pod 는 다시 생기는 걸 봤습니다. 서비스는 Deployment(무상태)나 StatefulSet(상태), 한 번 하고 끝나는 일은 Job 으로 만듭니다.

03 에이전트 이미지 만들기

코드를 상자 하나에

완료 목표 7주차의 상담 화면·에이전트 그래프·창구 서버를 담은 컨테이너 이미지 `onprem-agent:v1` 을 만들어 minikube 노드에 넣고, 크기와 만드는 시간을 잰다.

그림 03 이미지 한 장 = 코드 + 의존성, 모델은 밖



× 이미지에 넣지 않는다
모델 가중치 Qwen3-4B(FP8) 4.4GB · 검색 임베더 bge-m3 + torch 2GB → 07 · 10장에서 PVC 로 따로. 넣으면 3GB 가 넘어 올리기 · 되돌리기가 다 느려진다.

층 순서가 캐시를 정한다
잘 안 바뀌는 것(의존성) 먼저, 자주 바뀌는 것(코드) 나중 → 코드만 고치면 설치 층은 캐시에서.

한 이미지 두 역할 실행 명령만 바꿔 상담 화면(`python scripts/web.py`)과 전체 시험(`make test-full`, 12장)을 함께 한다. **EKS 라면** ECR 에 push 하고 노드가 pull.

이미지에는 **코드와 의존성만** 넣는다. 모델 서버는 07장의 Pod 가 따로 맡고, 검색 임베더(bge-m3, torch 2GB)도 넣지 않는다 — 넣으면 이미지가 3GB 가 되어 올리고 되돌리는 시간이 다 늘어난다. 03~09장은 해시 임베더로 돌고(규칙 모델과 시험이 쓰는 그대로), 검색 임베더는 10장에서 붙인다. 한 이미지가 **실행 명령만 바꿔** 두 역할을 한다: 상담 화면(`python scripts/web.py`)과 전체 시험(`make test-full`, 12장).

실행

Ubuntu Bash 03-001

```
cat deploy/Dockerfile
bash deploy/scripts/03-build.sh v1
cat results/cloud/03_image.txt
```

1 이미지 설계도를 읽는다. 층(layer) 순서가 요점이다 — `pyproject.toml` 과 `src` 를 먼저 복사해 `pip install` 하고, 자주 바뀌는 `scripts` · `examples` · `data` 는 뒤에 복사한다. Docker 는 바뀌지 않은 층을 캐시에서 재사용하므로 코드만 고치면 두 번째 빌드는 설치를 건너뛴다. 마지막에 일반 사용자(`agent`)로 바꾸고 `HEALTHCHECK` 로 `/healthz` 를 찍는다.

2 만들고(`docker build`), 호스트 Docker 에서 한 번 띄워 `/healthz` 가 200 인지 보고, `minikube image load` 로 노드 안 창고에 넣는다. 시간과 크기를 `results/cloud/03_image.txt` 에 남긴다. 한 줄씩 읽기는 부록 A3.

3 결과를 읽는다. `build:` 초, `size:` 바이트, 층 수, `/healthz` 응답(`{"ok":true,"pod":"…"}`), `load into minikube:` 초.

관찰 `docker history onprem-agent:v1` 에서 가장 큰 층은 `pip install` 층이다(langgraph · langchain · fastapi · 의존성). 코드 층은 몇 MB 다. 두 번째 빌드(`bash deploy/scripts/03-build.sh v1` 을 바로 다시)는 설치 층을 캐시에서 가져와 수 초에 끝난다 — 그 시간도 같은 파일에 쌓이니 둘을 견준다.

장애·복구 `pip install` 에서 멈추면 WSL2 의 DNS 문제가 잦다 — `docker build --network=host` 로 다시. `minikube image load` 가 수 분 넘게 걸리면 `minikube -p agent image build -t onprem-agent:v1 -f deploy/Dockerfile .` 로 노드 안에서 직접 만든다(호스트 캐시는 못 쓴다).

정리 없음(이미지는 04장이 쓴다). 호스트 쪽 확인용 컨테이너 `agent-check` 는 스크립트가 지운다.

성공 확인 `minikube -p agent image ls | grep onprem-agent` 에 `docker.io/library/onprem-agent:v1`, 아래 표를 채웠다.

항목	결과 (2026-10-12, 노트북, 커밋 446d794)
이미지 크기 / 층 수	**208MB** / 17층 — `pip install` 층 383MB(압축 전), 코드 층은 다 합쳐 2MB 안팎
처음 빌드 시간 / 코드 그대로 두 번째 빌드 시간	**43초 / 1초** (캐시)
`/healthz` (호스트 Docker)	`{"ok":true,"pod":"da212bfccd4a","model":"rule"}`
minikube 에 넣는 시간	11초, 두 번째 6초

(두 번째는 코드를 바꾸지 않고 돌린 것이라 '코드만 바꾼' 경우보다 빠르다 — 코드를 고치면 `COPY` 층부터 다시 만들지만 설치 층은 여전히 캐시다.)

실제 클라우드에서는 (AWS)

이 워크북	AWS	다른 점
`docker build` (노트북)	CodeBuild 또는 GitHub Actions(OIDC 로 AWS 권한)	빌드는 CI 가 하고 사람은 하지 않는다
`minikube image load`	**ECR** 에 `docker push`, 노드가 IAM 역할로 pull	ECR 이 이미지 취약점 스캔 · 수명 주기 정책
`imagePullPolicy: Never`	`IfNotPresent` + 태그 대신 **digest** 고정	같은 태그가 다른 이미지를 가리키는 사고를 막는다(C11)
208MB · 층 캐시	같은 원칙 + 멀티 스테이지 · 빌드 캐시를 레지스트리에	이미지가 작을수록 노드 확장 · 롤백이 빠르다

연습 — 바꿔 보고 관찰하기

1. **어느 층이 다시 만들어지나.** `src/` 의 파일 하나에 주석 한 줄을 넣고 `bash deploy/scripts/03-build.sh v1-test` → 예측: 설치 층은 캐시, 수 초. 그다음 `pyproject.toml` 끝에 빈 줄 하나 → 예측: `pip install` 층부터 다시(분 단위). 두 시간을 적고 둘 다 되돌린다(`git checkout -- .`).
2. **가장 큰 층 찾기.** `docker history onprem-agent:v1 --format '{{.Size}}\t{{.CreatedBy}}' | sort -h | tail -3`. 예측 → `pip install` 층(383MB).
3. **root 로 도나?** `docker run --rm onprem-agent:v1 id`. 예측 → `uid=1000(agent)`. 왜 root 가 아니어야 하는지 한 줄로 적는다.

이해 확인

1. Docker 층 캐시가 깨지는 조건은? (가) 이미지 이름이 바뀔 때 (나) 그 층이나 그 앞 층의 입력이 바뀔 때 (다) 노트북을 재시작할 때 (라) 태그가 바뀔 때
2. 모델 가중치를 이미지에 넣지 않은 이유 두 가지는?
3. `imagePullPolicy: Never` 인 매니페스트를 이미지를 넣지 않은 실제 클러스터에 올리면?
4. 왜 `COPY src` 보다 `COPY pyproject.toml` + `pip install` 을 먼저 두나?

면접에서 이렇게 묻는다

Q. 컨테이너 이미지를 어떻게 작고 빠르게 만드나요?

slim 베이스, 잘 안 바뀌는 의존성 층을 먼저 · 자주 바뀌는 코드를 나중에, `.dockerignore` 로 결과물 · 캐시 제외, 필요하면 멀티 스테이지, 그리고 큰 자산(모델 가중치)은 이미지 밖(PVC · S3)으로 뺍니다. 제 에이전트 이미지는 208MB 이고 코드만 바꾸면 다시 빌드가 1초였습니다.

Q. latest 태그를 쓰면 뭐가 문제인가요?

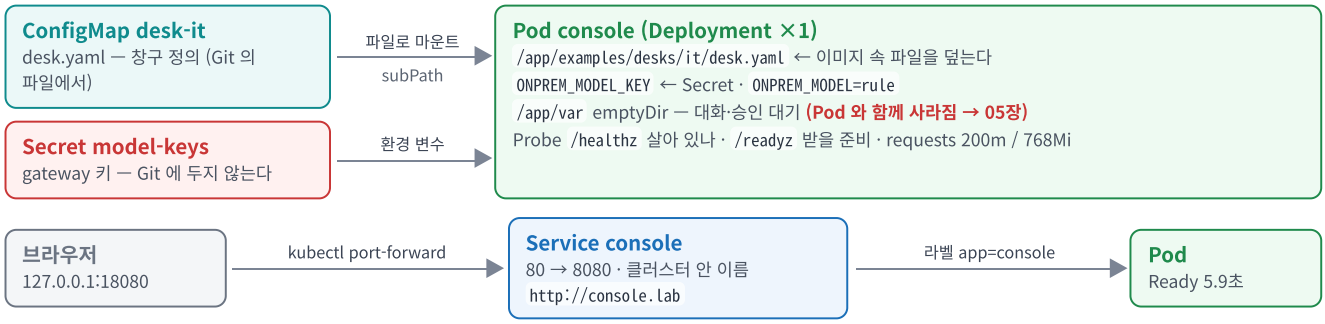
같은 이름이 날마다 다른 이미지를 가리킵니다. 실제로 3주차에 받은 SGLang `latest` 와 8주차에 노드가 받은 `latest` 가 달라서(36GB, 새 기본값) 설정이 어긋났고, 그 뒤로 digest 로 고정했습니다(C11).

04 설정과 비밀을 밖으로

ConfigMap · Secret · 상담 화면 Pod 하나

완료 목표 창구 정의(`desk.yaml`)를 ConfigMap 으로, 모델 키를 Secret 으로 밖에 두고, 상담 화면 Pod 하나를 띄워 브라우저로 연다. 이미지를 다시 만들지 않고 화면 문구가 바뀌는 것을 본다.

그림 04 **이미지는 그대로, 설정만 갈아 끼운다**



핵심 이미지는 그대로, 설정만 갈아 끼운다 — 7주차의 '창구 하나 = 폴더 하나'가 그대로 ConfigMap 으로. **EKS 라면** Secret 은 Secrets Manager + External Secrets, 설정은 ConfigMap 또는 Parameter Store.

7주차 프레임워크의 핵심이 여기서 그대로 쓰인다 — **창구 하나 = 폴더 하나**(`desk.yaml`) 라서, 창구 설정을 이미지 밖 ConfigMap 으로 뺄 수 있다. 실제 클라우드에서 '설정은 이미지에 굽지 않는다'는 원칙과 같다. 모델은 이 장과 05장에서 **규칙 모델**(GPU 없이, 시험 때 쓰는 것)이다 — 화면과 배포 흐름만 보고, 실제 모델은 07장에서 붙인다.

실행

Ubuntu Bash 04-001

```
bash deploy/scripts/04-config.sh
kubectl apply -f deploy/manifests/04-console.yaml
time kubectl -n lab rollout status deploy/console --timeout=180s
kubectl -n lab get pods,svc -o wide
kubectl -n lab port-forward svc/console 8080:80
```

1. ConfigMap `desk-it` 를 `examples/desks/it/desk.yaml` 파일에서, Secret `model-keys` 를 환경 변수 `GATEWAY_KEY` (없으면 5주차 개발용 키)에서 만든다. `--dry-run=client -o yaml | kubectl apply` 꼴이라 다시 돌려도 덮어쓴다. 부록 A4.
2. Deployment(상담 화면 Pod 의 설계도, 지금은 1개)와 Service(클러스터 안 이름)를 만든다. 매니페스트 해설은 아래.
3. Pod 가 준비(Ready)될 때까지의 시간을 잰다 — `readinessProbe` (`/readyz`)가 200 을 줘야 Ready 다.
4. Pod 의 IP·노드와 Service 의 ClusterIP 를 본다.
5. 내 컴퓨터의 8080 을 Service 의 80 에 잇는다. 브라우저에서 `http://127.0.0.1:8080/` (상담 화면) 과 `/admin` (관리자 화면). 끝낼 땐 Ctrl+C.

`deploy/manifests/04-console.yaml` 의 요점:

항목	뜻
`imagePullPolicy: Never`	03장에서 노드에 직접 넣은 이미지라 밖(Docker Hub)에서 받지 않는다. 빠뜨리면 `ErrImageNeverPull` / `ImagePullBackOff`
`env ONPREM_MODEL=rule`	어느 모델로 돌지. 07장에서 `sglang-4b` 로 바꾼다. `ONPREM_MODEL_URL` 은 07장 모델 서버의 Service 이름
`secretKeyRef model-keys/gateway`	키는 매니페스트에 적지 않고 Secret 에서 환경 변수로

항목	뜻
`volumeMounts … subPath: desk.yaml`	ConfigMap 의 파일 하나만 이미지 속 같은 자리에 덮는다(폴더째 덮으면 `prompt.md`·`server.py` 가 사라진다)
`emptyDir /app/var`	대화·승인 대기(SQLite). **Pod 가 사라지면 같이 사라진다** — 05장이 이걸 잰다
`livenessProbe /healthz` · `readinessProbe /readyz`	살아 있나(아니면 다시 띄움) · 받을 준비가 됐나(아니면 Service 가 요청을 안 보냄). `/readyz` 는 창구 정의를 읽을 수 있고 실제 모델이면 엔진에 닿아야 200
`resources requests/limits`	노드 6GB 중 이 Pod 몫(메모리 768Mi~1Gi — 처음 384Mi 로 적었다가 06장 측정 뒤 올림). 06장의 HPA 가 `requests.cpu` 를 기준 삼는다

관찰 — 설정만 바꾸기

Ubuntu Bash 04-002

```
sed 's/placeholder: 사번, 장비 번호, 오류 코드… 무엇이든 물어보세요/placeholder: (ConfigMap 에서 바꾼 문구) 무엇이든 물어보세요/' examples/desks/it/desk.yaml > /tmp/desk.yaml
bash deploy/scripts/04-config.sh /tmp/desk.yaml
kubectl -n lab rollout restart deploy/console && kubectl -n lab rollout status deploy/console
curl -s http://127.0.0.1:8080/api/desks | python3 -m json.tool | grep placeholder
```

1. 창구 정의의 입력칸 문구만 바꾼 사본을 만든다(저장소 파일은 그대로).
2. ConfigMap 을 그 사본으로 덮는다.
3. Pod 를 새로 띄운다 — ConfigMap 을 `subPath` 로 마운트하면 바뀐 내용이 자동으로 들어오지 않으므로 재시작이 필요하다(실제 서비스에서는 체크섬 주석으로 자동 재시작한다, 13장).
4. (5행의 port-forward 를 다시 켠 뒤) 상담 화면이 받는 창구 목록에 바뀐 문구가 보인다. 이미지는 그대로, 설정만 갈렸다. 끝나면 `bash deploy/scripts/04-config.sh` 로 되돌린다.

장애·복구 `port-forward` 가 `address already in use` 면 내 컴퓨터의 8080 을 누가 쓰고 있다(7주차 `scripts/web.py` 가 떠 있거나 지난 port-forward 가 남음) — `kubectl -n lab port-forward svc/console 18080:80` 으로 다른 포트에 잇고 브라우저는 `http://127.0.0.1:18080/`. · Pod 가 `CrashLoopBackOff` 면 `kubectl -n lab logs deploy/console --previous` 로 죽기 직전 출력을 본다. 흔한 원인: ConfigMap 의 `desk.yaml` 이 잘못돼 창구를 못 읽음(`/readyz` 503) · Secret 이 없음(`CreateContainerConfigError`). `kubectl -n lab describe pod -l app=console` 의 Events 가 어느 쪽인지 말해 준다.

정리 Ctrl+C 로 port-forward 를 끊는다. Deployment 는 05장이 쓰니 둔다.

성공 확인 **3행이 `successfully rolled out`, 브라우저에서 IT 헬프데스크 창구로 "VPN-809 오류가 떠요" 를 물으면 규칙 모델이 규정을 찾아 답한다(관리자 화면에 모델 `rule`). 표를 채운다.**

항목	결과 (2026-10-12, 노트북)
apply → Ready 시간 (3행 `real`)	**5.9초** (이미지가 이미 노드에 있어 받기 0초). 첫 시도의 port-forward 는 8080 충돌로 실패 → 18080 으로 열어 브라우저 확인
Pod 의 메모리 (`kubectl -n lab top pod` — metrics-server 애드온이 있을 때)	06장에서 metrics-server 를 켠 뒤 잰다
ConfigMap 문구 바꾸기 → 화면 반영 (04-002)	**됨** — 재시작 뒤 `/api/desks` 의 IT 창구 입력칸이 `(ConfigMap 에서 바꾼 문구) 무엇이든 물어보세요` 로. 이미지는 그대로. 재시작(rollout restart → 완료) 수 초

실제 클라우드에서는 (AWS)

이 워크북	AWS	다른 점
ConfigMap `desk-it`	ConfigMap 그대로, 또는 SSM Parameter Store	설정 변경을 Git 리뷰로(GitOps)

이 워크북	AWS	다른 점
Secret `model-keys` (base64)	**Secrets Manager** + External Secrets Operator 또는 Secrets Store CSI 드라이버	값은 AWS 에, 클러스터엔 참조만. etcd 는 **KMS 봉투 암호화**
환경 변수로 키 주입	IRSA / EKS Pod Identity 로 **AWS 권한 자체를** Pod 에	키를 아예 안 쓰는 방향
`/healthz` · `/readyz` Probe	같다 + ALB 대상 그룹 헬스체크	LB 헬스체크와 Probe 를 같은 경로로 맞춘다

연습 — 바꿔 보고 관찰하기

1. **망가진 설정을 넣으면?** `desk.yaml` 사본의 들여쓰기를 일부러 깨서 `04-config.sh /tmp/bad.yaml` → `kubectl -n lab rollout restart deploy/console`. 예측 → 새 Pod 가 `0/1` (`/readyz` 503), **옛 Pod 는 그대로 손님을 받는다**(롤링 업데이트가 멈춤). 되돌리기: `bash deploy/scripts/04-config.sh && kubectl -n lab rollout restart deploy/console`.
2. **Secret 이 없으면?** `kubectl -n lab delete secret model-keys` 뒤 재시작. 예측 → `CreateContainerConfigError`. 다시 `04-config.sh`.
3. **Secret 은 암호화일까?** `kubectl -n lab get secret model-keys -o jsonpath='{.data.gateway}' | base64 -d`. 예측 → 키가 그대로 보인다. base64 는 인코딩일 뿐이다.
4. **readiness 와 liveness 의 차이.** 1번 상태에서 `kubectl -n lab get pods -w` 로 RESTARTS 를 본다. 예측 → readiness 실패는 재시작하지 않고 Service 에서만 뺀다.

이해 확인

1. Kubernetes Secret 의 값은 기본적으로 어떻게 저장되나? (가) AES 로 암호화 (나) base64 인코딩 (다) 해시 (라) 저장하지 않음
2. `subPath` 로 마운트한 ConfigMap 을 바꾸면 Pod 안 파일이 바로 바뀌나?
3. liveness 가 실패할 때와 readiness 가 실패할 때 쿠버네티스는 각각 무엇을 하나?
4. 창구 정의를 ConfigMap 으로 뺀 덕에 할 수 있게 된 일은?

면접에서 이렇게 묻는다

Q. 설정과 비밀은 어떻게 관리하나요?

설정은 이미지에 굽지 않고 ConfigMap 으로 빼서 이미지를 다시 만들지 않고 바꿉니다 — 제 에이전트는 창구 하나가 `desk.yaml` 하나라 창구 설정이 그대로 ConfigMap 이 됩니다. 비밀은 Git 에 두지 않고, 쿠버네티스 Secret 은 base64 일 뿐이라 운영에선 Secrets Manager + External Secrets, etcd KMS 암호화, RBAC 로 읽을 수 있는 사람을 줄입니다. 가능하면 IRSA 로 키 자체를 없앱니다.

Q. Probe 세 가지를 설명해 주세요

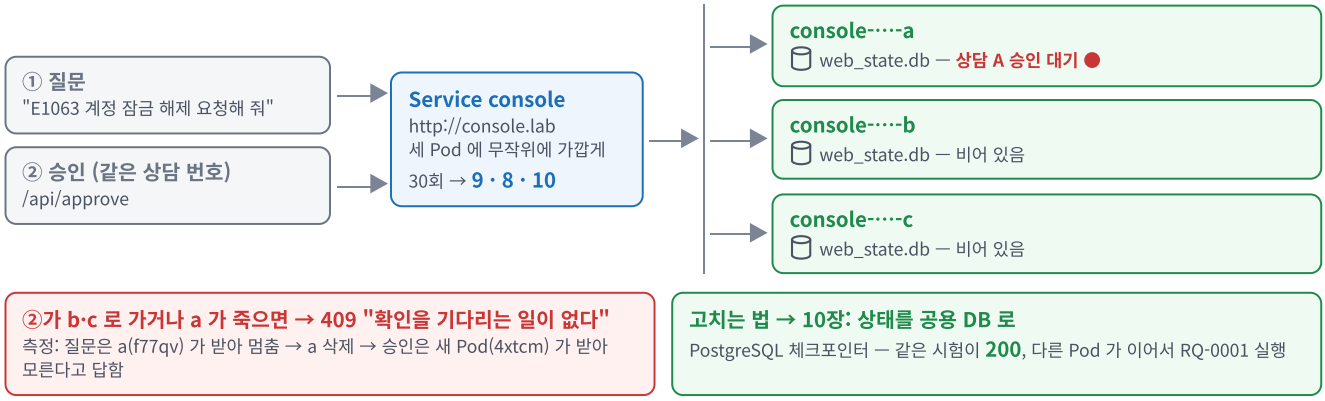
startup 은 처음 뜰 때까지 기다려 주는 것, liveness 는 살아 있나(아니면 재시작), readiness 는 받을 준비가 됐나(아니면 Service 에서 뺌)입니다. 실제로 모델 서버의 startupProbe 를 10분으로 두었더니 첫 시작의 커널 컴파일 중에 서버를 죽여 무한 반복했고, `/health` 가 토큰을 생성해 1초를 넘기는 바람에 기본 timeout 1초로는 영원히 Ready 가 안 됐습니다(C11). Probe 는 실제로 재서 정해야 합니다.

05 나눠 주기와 무상태

상담 화면 ×3, 로드 밸런서, Pod 를 죽여 보기

완료 목표 상담 화면 Pod 를 3개로 늘려 Service 가 요청을 나눠 주는 것을 세고, 승인을 기다리는 상담 중에 그 Pod 를 죽여 **상태가 Pod 안에 있으면 무슨 일이 생기는지** 본다.

그림 05 Service 는 라벨이 같은 Pod 전부에 돌아가며 보낸다



임시 처방 `sessionAffinity: ClientIP` — 같은 사람을 같은 Pod 로. 그 Pod 가 죽으면 역시 잃는다. **EKS 라면** ALB 의 sticky session 이 같은 처방이고 같은 한계를 가진다.

잰다 — 결과를 보기 전에 예상을 적는다 로드 밸런서: 30번 부르면 세 Pod 가 10번 안팎씩 받을 것이다(완전히 같지는 않다 — Service 는 무작위에 가깝다). 무상태: 04장의 설계대로면 대화·승인 대기가 **Pod 마다 따로**(emptyDir 의 SQLite) 있으므로, 질문을 받은 Pod 가 죽거나 승인 요청이 다른 Pod 에 가면 이어지지 않을 것이다 — 즉 **지금 상담 화면은 무상태가 아니다.** 이걸 눈으로 확인하는 게 이 장이고, 고치는 것(대화 저장을 공용 DB 로)이 10장이다.

실행

Ubuntu Bash 05-001

```
kubectl -n lab scale deploy/console --replicas=3 && kubectl -n lab rollout status deploy/console
kubectl -n lab get endpoints console
bash deploy/scripts/05-lb.sh 30
bash deploy/scripts/05-kill.sh
kubectl -n lab get pods -l app=console
```

1. 복제 수를 3으로. 같은 이미지·같은 설정의 Pod 가 둘 더 뜬다.
2. Service 뒤에 매달린 Pod 주소 세 개. 이 목록이 로드 밸런서의 '대상 그룹'이다.
3. 클러스터 안에 curl Pod 를 띄워 `http://console.lab/healthz` 를 30번 부르고, 답 머리말 `X-Served-By`(답한 Pod 이름)를 센다. `kubectl port-forward` 는 Pod 하나에 고정되므로 쓰지 않는다. 결과 `results/cloud/05_lb.txt`. 부록 A5.
4. ① 상담 번호 하나로 "E1063 계정 잠금 해제 요청해 줘…" 를 보내 담당자 확인(approval 사건)에서 멈추게 하고 어느 Pod 가 받았는지 적는다 → ② 그 Pod 를 지운다 → ③ 같은 상담 번호로 승인을 보낸다. 결과 `results/cloud/05_kill.txt`. 부록 A5.
5. 지운 Pod 자리에 새 Pod 가 떠 있다(Deployment 가 3개를 지킨다).

관찰 4행에서 승인이 `HTTP 409 확인을 기다리는 일이 없다` 로 끝나면 예상대로다 — 승인 대기가 죽은 Pod 의 SQLite 에만 있었다. Pod 를 지우지 않아도 승인 요청이 다른 Pod 에 가면(세 개 중 둘) 같은 409 다. 실제 클라우드에서 '앱 서버는 무상태여야 한다'는 말의 뜻이 이것이다: 서버 안에 남는 것이 있으면 늘리지도, 바꾸지도, 죽이지도 못한다.

장애·복구 당장의 임시 처방은 **같은 사람을 같은 Pod 로** 보내는 것이다: `kubectl -n lab patch svc console -p '{"spec":{"sessionAffinity":"ClientIP"}}'` . 브라우저 한 대로 쓰는 동안은 이어지지만, 그 Pod 가 죽으면 역시 잃는다(05-kill 로 확인). 제대로 된 처방은 10장 — 대화·승인 대기를 PostgreSQL 체크포인터로 옮겨 어느 Pod 가 받아도 같은 상담을 잇는다. 확인 뒤 되돌린다: `kubectl -n lab patch svc console -p '{"spec":{"sessionAffinity":"None"}}'` .

정리 `kubectl -n lab scale deploy/console --replicas=1` (06장이 1 에서 시작한다).

성공 확인 **`results/cloud/05_lb.txt` 에 Pod 이름 셋과 횟수, `05_kill.txt` 에 지운 Pod 이름과 승인 응답. 표를 채운다.**

항목	예상	결과 (2026-10-12, 노트북)
30회 요청의 Pod 별 횟수	10 안팎 × 3	**9 · 8 · 10** (`f77qv` · `nhzqj` · `nzsdm`) — 셋이 27회고 **3회는 Pod 이름 없이** 돌아왔다. 측정기가 HTTP 코드를 안 찍어 원인을 못 가렸다(1회용 Pod 의 첫 요청이 DNS 전에 나갔을 가능성). 스크립트가 코드를 같이 찍게 고쳐 06장에서 다시 세니 `26 200 …` — 이번엔 4회가 없고 오류도 없었다. 원인은 `kubectl run -i` 가 붙기 전의 앞 줄을 잃는 것(D45) — 요청은 30번 다 갔다. logs 로 읽게 고치니 **30/30**(06-002)
승인 대기 중 Pod 삭제 → 같은 상담 번호 승인	409 (이어지지 않음)	**409 확인을 기다리는 일이 없다** — 질문은 `f77qv` 가 받아 담당자 확인에서 멈췄고, 그 Pod 를 지운 뒤 승인은 새 Pod `4xtcm` 이 받아 모른다고 답함
새 Pod 가 다시 3개가 되기까지	수 초	승인 요청 시점(삭제 3초 뒤)에 이미 새 Pod 가 답했다
결론	지금 상담 화면은 무상태가 아니다 → 10장	**예상대로.** 승인 대기는 죽은 Pod 의 SQLite 에만 있었다

실제 클라우드에서는 (AWS)

이 워크북	AWS	다른 점
Service(ClusterIP) 가 Pod 에 나눠 줌	`type: LoadBalancer` → **NLB**(L4), Ingress → **ALB**(L7) — AWS Load Balancer Controller	ALB 는 경로 · 호스트 규칙 · 헬스체크 · 인증서
`sessionAffinity: ClientIP`	ALB **sticky session**(쿠키)	같은 처방, 같은 한계 — 그 대상이 죽으면 잃는다
상태를 Pod 밖으로(10장)	세션 · 상태를 ElastiCache · RDS · DynamoDB 에	앱 서버는 무상태 → 오토스케일링 · 스팟 · 롤링 배포가 안전

연습 — 바꿔 보고 관찰하기

1. **sticky 를 켜면 나눠 주기는?** `kubectl -n lab patch svc console -p '{"spec":{"sessionAffinity":"ClientIP"}}'` 뒤 `bash deploy/scripts/05-lb.sh 30` . 예측 → 30회가 한 Pod 로(curl Pod 하나는 IP 하나). 끄고 다시 재서 비교.
2. **Pod 가 바뀌는 순간.** 한 창에서 `kubectl -n lab get endpoints console -w` , 다른 창에서 `kubectl -n lab delete pod -l app=console --wait=false` . 예측 → 주소가 빠졌다가 새 Pod 가 **Ready 가 된 뒤에** 들어온다.
3. **409 다시 보기(10장 전에).** sticky 를 켠 채 `05-kill.sh` . 예측 → 그래도 409 — 질문을 받은 Pod 를 죽였으니까.

이해 확인

1. Service 가 요청을 보낼 Pod 를 고르는 기준은? (가) Pod 이름 (나) 라벨 셀렉터 (다) 노드 이름 (라) 생성 시간
2. 로드 밸런싱을 `kubectl port-forward` 로 재면 안 되는 이유는?
3. 05장의 409 는 정확히 무엇 때문인가?

4. sticky session 으로 409 를 '고칠' 수 없는 경우는?

면접에서 이렇게 묻는다

Q. 무상태(stateless) 앱이 왜 중요한가요?

어느 서버가 받아도 같은 답이 나와야 늘리고, 바꾸고, 죽일 수 있기 때문입니다. 제 에이전트를 3개로 늘리자 로드 밸런서는 9 · 8 · 10 으로 잘 나눴는데, 승인 대기 중에 그 Pod 를 죽이자 승인이 409 로 끊겼습니다 — 승인 대기가 Pod 안 SQLite 에 있었기 때문입니다. PostgreSQL 체크포인터로 옮기자 같은 시험이 200 이 됐습니다. Pod 를 늘렸다고 늘린 게 아니었습니다.

Q. L4 와 L7 로드 밸런서의 차이는?

L4(NLB)는 TCP 연결 단위로 나누고, L7(ALB)은 HTTP 를 읽어 경로 · 호스트 · 헤더로 나눕니다. 제 상담 화면은 답을 SSE 로 수십 초 흘려서, L7 앞단에선 유휴 타임아웃과 버퍼링을 꺼야 했습니다(08장 `proxy-buffering: off`, 600초).

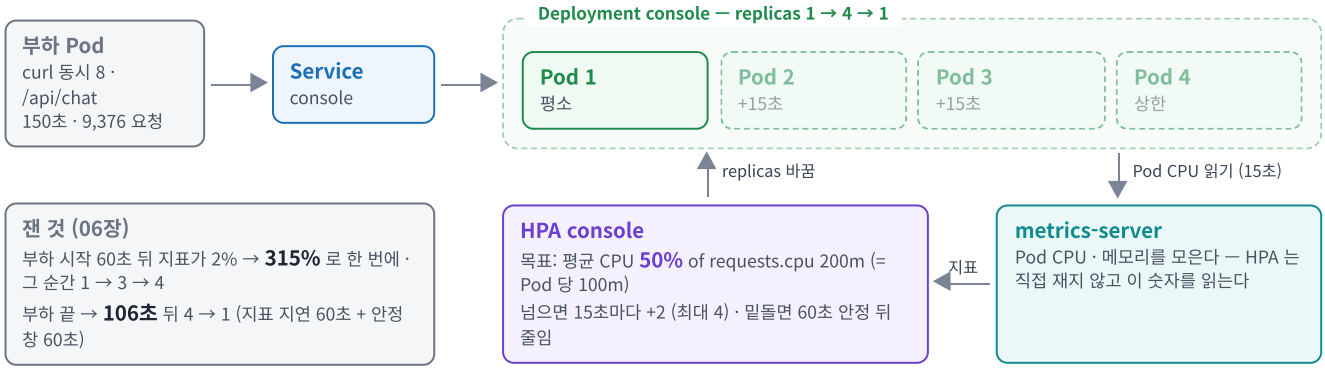
측정 원본: reports/cloud_w8.md.

06 자동으로 늘리고 줄이기

metrics-server · HPA · 부하

완료 목표 상담 화면 Pod 를 CPU 사용률로 1 → 4 개까지 자동으로 늘리는 HPA 를 걸고, 부하를 주어 늘어나는 과정과 부하가 끝난 뒤 줄어드는 과정을 10초 간격으로 적는다.

그림 06 HPA 는 '평균 CPU 가 목표를 넘으면 Pod 를 더' — 측정은 metrics-server 가 한다



없는 것 노드 자동 확장 — 노트북 한 대. **EKS 라면** Pod 가 Pending 이 되면 Cluster Autoscaler / Karpenter 가 같은 원리로 EC2 노드를 더한다(HPA = 가로로 Pod, Karpenter = 노드).

HPA 는 `requests.cpu` 를 100% 로 본다. 04장의 상담 화면 Pod 는 `requests.cpu: 200m` 이므로 목표 50% 는 Pod 당 100m — 규칙 모델 질문 하나가 그래프 + 창구 서버(MCP)를 거쳐 수십 ms 의 CPU 를 쓰니, 동시 8개를 계속 보내면 넘는다. **예상:** 30~60초 안에 Pod 가 1 → 3~4 로 늘고, 부하가 끝나고 60초 안정 뒤 1 로 돌아온다. 처음 10~20초는 HPA 가 `<unknown>` 을 보인다(metrics-server 가 첫 수치를 모으는 시간).

실행

Ubuntu Bash 06-001

```
minikube -p agent addons enable metrics-server
kubectl -n kube-system rollout status deploy/metrics-server --timeout=120s
kubectl -n lab top pod
kubectl apply -f deploy/manifests/06-hpa.yaml
kubectl -n lab get hpa console
bash deploy/scripts/05-lb.sh 30
bash deploy/scripts/06-load.sh 150 8
```

1. metrics-server 를 켠다 — 노드·Pod 의 CPU·메모리를 15초마다 모아 `kubectl top` 과 HPA 에 준다. 실제 클라우드의 CloudWatch 지표 수집에 해당.
2. 뜰 때까지 기다린다.
3. 상담 화면 Pod 의 지금 CPU·메모리(04장 표의 빈칸이 여기서 채워진다). 처음 1분은 `metrics not available yet` 일 수 있다.
4. HPA 를 만든다. 매니페스트 해설은 위 그림.
5. `TARGETS` 가 `<unknown>/50%` 에서 곧 `n%/50%` 로 바뀐다.
6. 05장에서 30회 중 3회가 Pod 이름 없이 돌아온 것을, HTTP 코드를 찍게 고친 측정기로 다시 센다(replicas 는 1 이라 전부 한 Pod 가 받는다 — 빠진 요청이 있는지만 본다).
7. 부하 150초(동시 8)를 걸고 10초마다 HPA 의 현재 %, 목표 Pod 수, 준비된 Pod 수를 적는다. 부하가 끝난 뒤 90초를 더 보아 줄어드는 것까지 적는다. 결과 `results/cloud/06_hpa.txt`. 부록 A6.

관찰 표에서 세 시점을 찾는다 — 현재 % 가 50 을 처음 넘은 때, Pod 수가 처음 늘어난 때(그 차이가 HPA 의 반응 시간), 부하 종료 뒤 Pod 가 1 로 돌아온 때(60초 안정 창 + 측정 주기). `kubectl describe hpa console` 의 Events 에 `SuccessfulRescale … above target` 과 `… below target` 이 보인다.

장애·복구 `TARGETS` 가 계속 `<unknown>` 이면 metrics-server 가 Pod 를 못 읽는 것이다 — `kubectl -n kube-system logs deploy/metrics-server | tail` 에 `x509` 가 보이면 minikube 애드온은 이미 `--kubelet-insecure-tls` 를 넣어 주지만, 아니면 `kubectl -n kube-system edit deploy metrics-server` 로 그 인자를 더한다. Pod 가 4 까지 늘지 않고 2~3 에서 멈추면 부하보다 Pod 가 빨리 처리하는 것이다(좋은 일) — 7행의 동시 수를 16 으로 올려 본다.

정리 `kubectl delete -f deploy/manifests/06-hpa.yaml` (HPA 를 두면 07장에서 모델 서버가 노드 메모리를 쓰는 동안 상담 화면이 늘어나 겹칠 수 있다). Pod 는 1 로 둔다.

성공 확인 **`results/cloud/06_hpa.txt` 에 Pod 수가 1 보다 커진 줄과 다시 1 이 된 줄이 모두 있다.**

항목	예상	결과 (날짜, 노트북)
상담 화면 Pod 의 평시 CPU · 메모리 (`top pod`)	수 m · 200~300 Mi	**4m · 670~764Mi** — 메모리가 예상의 두 배 넘게 크다(langchain·langgraph·창구 서버 적재). `requests.memory: 384Mi` 는 작게 잡은 것 — 4개면 2.9GB, 07장 전에 `requests` 를 768Mi 로 올린다
05장 다시 세기: 30회 중 HTTP 200 과 Pod 이름이 찍힌 수	30/30	26/30(두 번째 측정기) → **30/30**(logs 로 읽는 세 번째 측정기, 06-002) — 빠진 건 측정기였다(D45)
50% 를 처음 넘은 뒤 Pod 가 늘기까지	15~30초	부하 시작 09:23:19 → 지표는 60초 동안 2% 에 머물다 09:24:21 에 **315%** 로 한 번에 뛰며 그 순간 Pod 3 → 10초 뒤 4. 즉 **지표가 약 60초 늦게** 오고, 오자마자 HPA 는 바로 늘렸다(한 단계 +2, 다음 +1 = 상한 4)
부하 중 최대 Pod 수	3~4	**4**(상한). 150초에 9,376 요청(초당 62), Pod 당 CPU 630m(315%)
부하 끝 → 1 로 돌아오기까지	60~90초	**못 봄** — 부하가 09:25:49 에 끝났는데 관찰이 09:27:16 까지였고 지표는 아직 115%(늦게 오는 지표가 내려오는 중). 줄어드는 데는 '지표 지연 60초 + 안정 창 60초' 이상이 든다. 스크립트가 부하 뒤 240초를 보게 고쳐 07장 전에 한 번 더(아래 06-002)

Ubuntu Bash (줄어드는 것까지 보기 — 07장 전에) `06-002`

```
kubectl apply -f deploy/manifests/06-hpa.yaml
bash deploy/scripts/06-load.sh 90 8 240
kubectl delete -f deploy/manifests/06-hpa.yaml
```

1~3 같은 HPA 로 부하 90초 뒤 240초를 더 본다. 표에 '부하 끝 → 1' 시간을 채운다.

06-002 항목	결과 (2026-10-12, 노트북)
부하 끝 → Pod 1 로	**106초** — 부하 09:48:51~09:50:21, 지표 38% 가 09:51:26 에 처음 보이고(지연 약 60초) 60초 안정 창 뒤 09:52:07 에 4 → 1. Events `New size: 1; reason: All metrics below target`
늘어나기(다시)	09:50:24 에 3, 09:50:45 에 4 — 첫 측정과 같은 모양. 부하 90초에 4,152 요청. 처음 60초의 빈 지표는 직전 `apply` (메모리 768Mi)로 Pod 가 새로 떠서 `pods might be unready`
05장 다시 세기(logs 로 읽는 측정기)	**30/30, 전부 200** — D45 확인. 빠졌던 3~4회는 요청이 아니라 측정기였다

실제 클라우드에서는 (AWS)

이 워크북	AWS	다른 점
metrics-server + HPA (CPU 50%)	같다	그대로 쓴다
노드 자동 확장 **없음**	**Karpenter** 또는 Cluster Autoscaler	Pod 가 Pending 이면 노드를 더하고, 비면 줄인다

이 워크북	AWS	다른 점
CPU 기준	**KEDA** — 큐 길이 · 요청 수 · Prometheus 지표 기준	LLM 서버는 CPU 가 아니라 대기열 · GPU 사용률로 늘린다

연습 — 바꿔 보고 관찰하기

1. **목표를 낮추면.** `06-hpa.yaml` 의 `averageUtilization` 을 20 으로 → `06-load.sh 90 8 240`. 예측 → 더 일찍 4 에 닿는다.
2. **requests 를 올리면.** `04-console.yaml` 의 `requests.cpu` 를 400m 으로 → 같은 부하. 예측 → 사용률 % 가 절반 → 덜 늘어난다. **HPA 의 % 는 requests 기준**이라는 걸 몸으로 확인.
3. **줄이기를 천천히.** `scaleDown.stabilizationWindowSeconds: 300`. 예측 → 4 → 1 이 5분 넘게. 왜 기본값이 300초인지 한 줄로 적는다.
4. **노드가 모자라면.** `maxReplicas: 10` 으로 올리고 동시 16 부하. 예측 → 몇 개는 `Pending`(노드 메모리). 실제 클라우드라면 여기서 Karpenter 가 노드를 더한다.

이해 확인

1. HPA 의 'CPU 50%' 는 무엇의 50% 인가? (가) 노드 CPU (나) limits.cpu (다) requests.cpu (라) 코어 1개
2. HPA 를 만든 직후 `TARGETS` 가 `<unknown>` 인 이유는?
3. 늘리기는 바로 하고 줄이기는 기다리는 이유는?
4. Qwen3-4B 모델 서버를 CPU 기준 HPA 로 늘리면 안 되는 이유 두 가지는?

면접에서 이렇게 묻는다

Q. HPA 와 Cluster Autoscaler(Karpenter)는 어떻게 다른가요?

HPA 는 Pod 수를(가로로), Karpenter 는 노드 수를 늘립니다. HPA 가 Pod 를 늘렸는데 자리가 없으면 Pending 이 되고, 그걸 보고 Karpenter 가 노드를 더합니다. 제 실습에서 HPA 는 부하 시작 60초 뒤 지표가 315% 로 뛰자 1 → 3 → 4 로 늘렸고, 부하가 끝난 뒤 106초(지표 지연 60초 + 안정 창 60초)에 1 로 돌아왔습니다.

Q. LLM 서버는 어떻게 오토스케일링하나요?

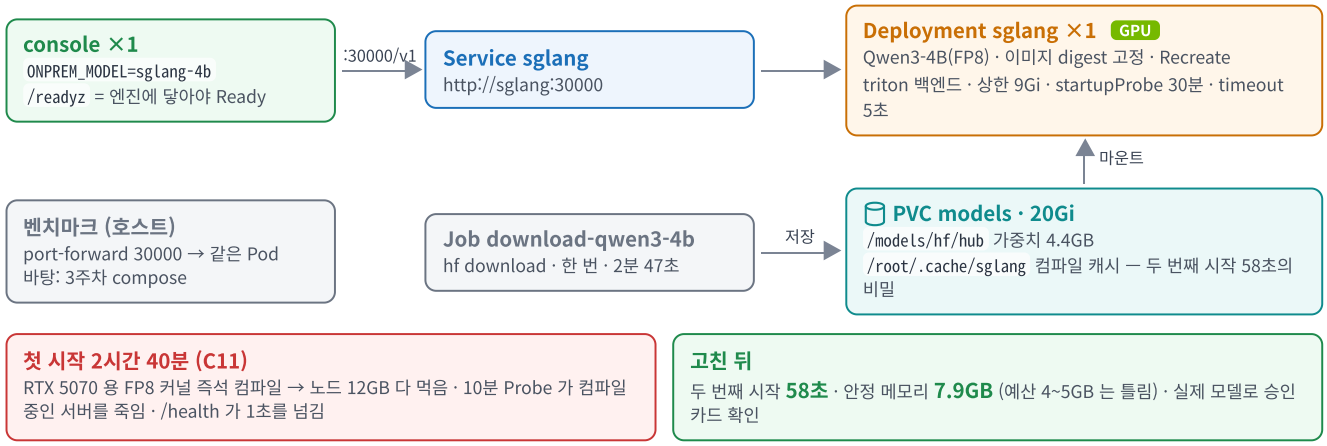
CPU 는 낮게 나오고, 새 인스턴스는 뜨는 데 오래 걸립니다(제 모델 서버는 캐시가 있어도 58초, 첫 시작은 수십 분). 그래서 대기열 길이 · 첫 글자 시간 · GPU 사용률 같은 지표로 KEDA 를 쓰고, 최소 대수를 두고 미리 늘립니다.

07 모델 서버를 클러스터 안으로

GPU Pod · PVC · 상담 화면을 실제 모델로

완료 목표 01장 판정 A 에 따라 SGLang(Qwen3-4B(FP8))을 GPU 를 요구하는 Pod 로 올리고, 가중치는 PVC 에 두며, 상담 화면이 규칙 모델 대신 이 모델 서버를 쓰게 바꾼다. 그리고 **Pod 로 올려서 느려졌는지** 3주차 측정기로 호스트 Docker 와 견준다.

그림 07 모델 서버는 Pod 하나, 가중치는 PVC, 상담 화면은 이름으로 붙는다



옮길 큰 파일이 둘이다 모델 서버를 띄우려면 **프로그램**과 **재료**가 모두 노드 안에 있어야 한다. 프로그램은 SGLang 이미지(CUDA·PyTorch 포함, 10GB 급) — 3주차에 노트북 Docker 에 받아 두었지만 minikube 노드는 자기 창고가 따로라 `minikube image load` 로 복사한다(2행). 재료는 Qwen3-4B(FP8) 가중치(모델이 배운 내용이 든 숫자 파일, 4.4GB) — 1회용 Job 이 Hugging Face 에서 PVC(클러스터 안 저장 공간)로 한 번 받아 두면, 모델 서버 Pod 가 죽었다 떠도 다시 받지 않는다(4행). 03장에서 에이전트 이미지에 모델을 넣지 않은 것과 같은 이유로, 프로그램과 재료를 떼어 둔다. 비유: 식당을 열려면 주방 설비(이미지)와 재료(가중치)가 다 건물(노드) 안에 있어야 하고, 설비를 바꿀 때 재료까지 다시 사 오지 않는다.

메모리 먼저 01장에서 노드를 `--memory=6g` 로 만들었다 — docker 드라이버에서 이 숫자는 노드 컨테이너의 Docker 메모리 상한이다. SGLang 은 GPU 외에 호스트 RAM 을 3~4GB 쓰므로 상담 화면과 합치면 6GB 를 넘길 수 있다. 클러스터를 다시 만들지 않고 **`docker update` 로 상한만 올린다**(1행) — WSL2 의 15GB 중 10GB 를 노드에, 나머지는 벤치마크·브라우저 몫. 02장 2행에서 본 노드의 `Allocatable memory` 가 호스트 전체(15Gi 근처)로 보였다면 kubelet 은 이 상한을 모른다는 뜻이다 — 스케줄러는 막지 않고, 넘으면 노드 컨테이너 안에서 OOM 으로 죽는다. 그래서 이 장은 **상담 화면 1개 + 모델 서버 1개**만 띄운다.

실행

Ubuntu Bash — 07-001

```
docker update --memory=10g --memory-swap=10g agent && docker inspect agent --format '{{.HostConfig.Memory}}'
docker images lmsysorg/sglang:latest --format '{{.Size}}'
kubectl apply -f deploy/manifests/07-sglang.yaml
kubectl apply -f deploy/manifests/07-download.yaml && time kubectl -n lab wait --for=condition=complete job/download-qwen3-4b --timeout=1800s
kubectl -n lab logs job/download-qwen3-4b | tail -3
time kubectl -n lab rollout status deploy/sglang --timeout=1800s
kubectl -n lab logs deploy/sglang | grep -E "cache|graph|Uvicorn|ready" | tail -5
kubectl -n lab set env deploy/console ONPREM_MODEL=sglang-4b && kubectl -n lab rollout status deploy/console
kubectl -n lab port-forward svc/console 18080:80
```

1 노드 컨테이너(이름 `agent`)의 메모리 상한을 10GB 로. 출력 `10737418240`. 클러스터·Pod 는 그대로 산다. `minikube stop/start` 를 하면 노드 컨테이너가 다시 만들어져 6g 로 돌아가므로 start 뒤 다시 건다.

2 3주차에 받아 둔 SGLang 이미지의 크기만 본다(10GB 급). 처음엔 03장처럼 `minikube image load` 로 노드에 복사하려 했는데 **WSL2 가 통째로 재시작됐다**(C10) — 10GB 를 임시 파일로 풀며 메모리를 넘긴 것으로 보인다. 그래서 이 이미지는 **노드가 Docker Hub 에서 직접 받는다**(`imagePullPolicy: IfNotPresent` , 3행의 apply 가 받기 시작). 인터넷 속도에 달렸고 6행의 기다림에 포함된다.

3 PVC(20Gi) + 모델 서버 Deployment + Service 를 만든다. 가중치가 아직 없으니 Pod 는 startupProbe 를 기다리며 실패를 거듭한다 — 괜찮다, 4행이 끝나면 다음 재시작에 뜬다. (`HF_HUB_OFFLINE=1` 이라 인터넷에서 받으려 하지 않는다.)

4 가중치 내려받기 Job. `python:3.12-slim` 에 `huggingface_hub` 를 깔고 `hf download Qwen/Qwen3-4B-FP8` 를 PVC 로. 4.4GB 라 인터넷 속도에 달렸다(100Mbps 면 6분). 시간을 적는다. 호스트 `~/.cache/huggingface` 에 이미 있는 파일을 재활용하는 길은 장애·복구.

5 `du -sh` 로 받은 크기.

6 모델 서버가 Ready 될 때까지 — 이미지 받기(10GB) + 가중치 읽기 + CUDA 그래프 준비. 시간을 적는다. 다른 창에서 `kubectl -n lab describe pod -l app=sglang | tail -5` 로 `Pulling` → `Pulled` 를 본다.

7 접두사 캐시(`enable-cache-report`)·CUDA 그래프 줄이 보이면 3주차와 같은 설정이다.

8 상담 화면의 모델을 `sglang-4b` 로 바꾼다 — `ONPREM_MODEL_URL=http://sglang:30000/v1` (04장 매니페스트)로 붙는다. `/readyz` 가 모델 서버에 닿아야 Ready 가 되므로 6행이 먼저다.

9 브라우저 `http://127.0.0.1:18080/` — 이제 **실제 모델**이 답한다. IT 헬프데스크에 "E1063 계정 잠금 해제 요청해 줘. 비밀번호를 다섯 번 틀렸대요. 본인 확인은 끝났어요." 를 넣으면 담당자 확인 카드가 뜨고, 승인하면 요청이 실행된다. 관리자 화면에 모델 `sglang-4b` , 엔진 `닿음` .

잰다 — Pod 로 올려서 느려졌나 (서빙 측정기 그대로, 생각 모드 끔, 동시 1·4·8)

Ubuntu Bash 07-002

```
kubectl -n lab port-forward svc/sglang 30000:30000 &
python scripts/bench_serving.py configs/serving_cloud_pod.yaml
kill %1; kubectl -n lab scale deploy/sglang --replicas=0 && kubectl -n lab scale deploy/console --replicas=0
python scripts/bench_serving.py configs/serving_cloud_host.yaml --manage
kubectl -n lab scale deploy/sglang --replicas=1 && kubectl -n lab scale deploy/console --replicas=1
cat results/serving_cloud_pod.md results/serving_cloud_host.md
```

1~2 Pod 를 호스트의 30000 으로 잇고 잰다(결과 `results/serving_cloud_pod.md`). port-forward 가 한 겹 더 끼므로 첫 글자 시간에 몇 ms 가 얹힌다 — 이건 측정 경로의 비용이고 Pod 의 비용이 아니다. 둘을 가르려면 3행 뒤 `kubectl -n lab run` 으로 클러스터 안에서 재야 하는데, 이 장에서는 port-forward 로 재고 그 사실을 적는다.

3 GPU 는 하나다 — Pod 를 내려 GPU 를 비운다(상담 화면도 `/readyz` 가 실패하므로 같이).

4 바탕: 3주차 compose 의 `sglang-4b` 프로필을 `--manage` 로 띄워 같은 요청으로 잰다(결과 `results/serving_cloud_host.md`). 끝나면 측정기가 내린다.

5 되돌린다. 6행 두 표를 나란히 읽는다.

예상 첫 글자 시간(TTFT)·토큰당 시간(TPOT)·처리량 모두 **5% 안**에서 같다. (단, Pod 쪽은 C11 로 attention·FP8 행렬곱 백엔드가 triton 이고 호스트는 3주차의 옛 이미지·기본 백엔드라, 차이가 나면 'Pod 의 비용'이 아니라 '백엔드 차이'일 수 있다 — 호스트도 같은 인자로 돌리는 게 맞는 비교이며 `serving_cloud_host.yaml` 의 `SGLANG_EXTRA` 로 맞춘다.) GPU 계산은 같은 드라이버·같은 커널이고, 컨테이너 한 겹(Pod)은 CPU 쪽 오버헤드만 더한다. 접두사 캐시 적중률도 같아야 한다(같은 요청 묶음).

장애·복구 (가) Pod 가 `Pending` 에 `Insufficient nvidia.com/gpu` 면 01장의 장치 플러그인이 내려간 것 — `kubectl -n kube-system rollout restart ds/nvidia-device-plugin-daemonset` . (나) `OOMKilled` 또는 노드가 `NotReady` 면 메모리 — 1행의 상한을 12g 로, 또는 `--mem-fraction-static 0.80` 은 그대로 두고 상담 화면을 0 으로. (다) 가중치 내려받기가 느리면 호스트 캐시를 쓴다: `minikube -p agent mount ~/.cache/huggingface:/hf &` 를 켜 두고 Job 대신 `kubectl -n lab run cp --image=busybox --restart=Never --overrides='…hostPath /hf…' -- cp -r /hf/hub/models--Qwen--Qwen3-4B-FP8 /models/hf/hub/` — 9p 라 느리지만 인터넷보다 빠를 수 있다. (라) 6행이 900초를 넘기면 `kubectl -n lab logs deploy/sglang` 의 마지막 줄 — `CUDA out of memory` 면 노트북에서 다른 GPU 프로그램(3주차 compose)이 떠 있는 것이다. `docker compose -f serving/compose.yml --profile sglang-4b down` .

정리 모델 서버는 08장 이후에도 쓰니 둔다. 다른 장을 돌리는 동안 GPU 를 비우려면 `kubectl -n lab scale deploy/sglang --replicas=0`.

성공 확인 **브라우저에서 실제 모델이 답하고 담당자 확인이 뜬다. 아래 표를 채웠다.**

항목	예상	결과 (2026-10-12, 노트북)
SGLang 이미지 크기 / 노드가 받는 시간	10GB 급 / 인터넷 속도	노드 안에서 **36.4GB**(압축 풀린 크기) / 받기는 WSL 재시작 전후에 걸쳐 끝나 못 잼
가중치 내려받기(Job) 시간 / 크기	인터넷 속도 / 4.4GB	**2분 47초**(Job 완료까지, 두 번 실패 뒤 세 번째 Pod 가 성공 — 앞 둘은 WSL 재시작 때)
모델 서버 Ready 까지 (6행)	1~3분	**첫 시작: 약 2시간 40분의 사투**(C11) — FP8 커널 즉석 컴파일이 메모리를 넘겨 OOM 반복, 점검 시간 초과. 설정 네 가지를 바꿔 뜸. **두 번째 시작(캐시 있음): 58초**
노드 메모리: 모델 서버 혼자 안정 / 상담 화면과 함께	4~5GB(00장 예산)	**7.9GB / 11.1GB** — 예산이 틀렸다. 00장 예산표를 고침
상담 화면 → 모델 서버 전환 (8행) 뒤 실제 답	담당자 확인 카드	**됨** — "E1063 계정 잠금 해제…" 에 잠금 해제 요청 승인 카드(사번 E1063, 사유 '비밀번호 5회 오류로 잠김, 본인 확인 완료'). 관리자 화면: 모델 qwen3-4b, 주소 http://sglang:30000/v1, 생각 모드 켬
TTFT · TPOT · 처리량: Pod 대 호스트 Docker (동시 1·4·8)	5% 안	
접두사 캐시 적중률: Pod 대 호스트	같음	

실제 클라우드에서는 (AWS)

이 워크북	AWS	다른 점
노드 컨테이너 상한 12GB · 컨테이너 상한 9Gi	GPU 인스턴스(g5.xlarge 16GB RAM …) + Pod `limits.memory`	같은 원칙 — 상한이 없으면 노드 전체가 멈춘다 (C11 ①)
PVC `models` 에 가중치	**S3** → 초기화 컨테이너로 노드 로컬 NVMe, 또는 FSx for Lustre	여러 노드가 같은 가중치를 빨리 읽어야 한다
36GB 이미지를 노드가 받음	미리 받아 둔 AMI · Bottlerocket 데이터 볼륨 · SOCI 지연 로딩	콜드 스타트의 큰 몫이 이미지 받기
모델 서버를 직접 운영	SageMaker 엔드포인트 · Bedrock(관리형)	온프레미스 요구가 있으면 직접 운영

연습 — 바꿔 보고 관찰하기 (무겁다 — 하나씩, 끝나면 되돌린다)

1. **startupProbe 를 짧게.** `failureThreshold: 6`(1분)으로 바꿔 `kubectl apply`. 예측 → 캐시가 있으면 58초라 겨우 통과하거나 한 번 재시작. C11 ② 가 이 모양이었다.
2. **메모리 상한을 낮게.** `limits.memory: 6Gi`. 예측 → `OOMKilled`(안정 7.9GB). `kubectl -n lab describe pod -l app=sglang | grep -A3 "Last State"`.
3. **점검 대기를 1초로.** readinessProbe `timeoutSeconds: 1`. 예측 → 서버는 돌지만 `0/1` 영원히(C11 ④).
4. **모델 서버를 내리면 상담 화면은?** `kubectl -n lab scale deploy/sglang --replicas=0`. 예측 → console 의 `/readyz` 503 → Service 에서 빠져 화면 503. readiness 가 의존성까지 본다는 뜻. 되돌리고 다시 Ready 까지의 시간을 잰다.

이해 확인

1. 모델 서버 Deployment 에 `strategy: Recreate` 를 쓴 이유는? (가) 빠르게 배포하려고 (나) GPU 가 하나라 새 Pod 가 옛 Pod 와 동시에 뜰 수 없어서 (다) 데이터를 지우려고 (라) 기본값이라서
2. 첫 시작 2시간 40분과 두 번째 시작 58초의 차이를 만든 것은?
3. C11 에서 Pod 는 돌고 있는데 `0/1` 이었던 이유는?

4. 메모리 예산 '모델 서버 4~5GB' 가 틀린 것을 무엇으로 알았나?

면접에서 이렇게 묻는다

Q. LLM 을 쿠버네티스에 올리면서 가장 어려웠던 점은?

첫 시작입니다. 호스트 Docker 에선 몇 분이면 뜨던 모델 서버가 Pod 로는 2시간 40분 걸렸습니다. 원인은 연쇄였습니다 — 노트북 GPU 용 FP8 커널을 즉석 컴파일하며 메모리를 12GB 까지 먹어 제어부가 멈췄고, 10분짜리 startupProbe 가 컴파일 중인 서버를 죽였고, `/health` 가 실제로 토큰을 생성해 1초 timeout 을 넘겼습니다. 30초마다 메모리를 찍은 그래프로 '컴파일 중인가, 죽는 중인가'를 가렸고, 컴파일 없는 백엔드 · 컴파일 캐시 PVC · 상한 · Probe 를 고쳐 두 번째 시작 58초, 안정 7.9GB 로 만들었습니다. 교훈은 상한 없는 환경이 비용을 숨긴다는 것입니다.

Q. 콜드 스타트를 어떻게 줄이나요?

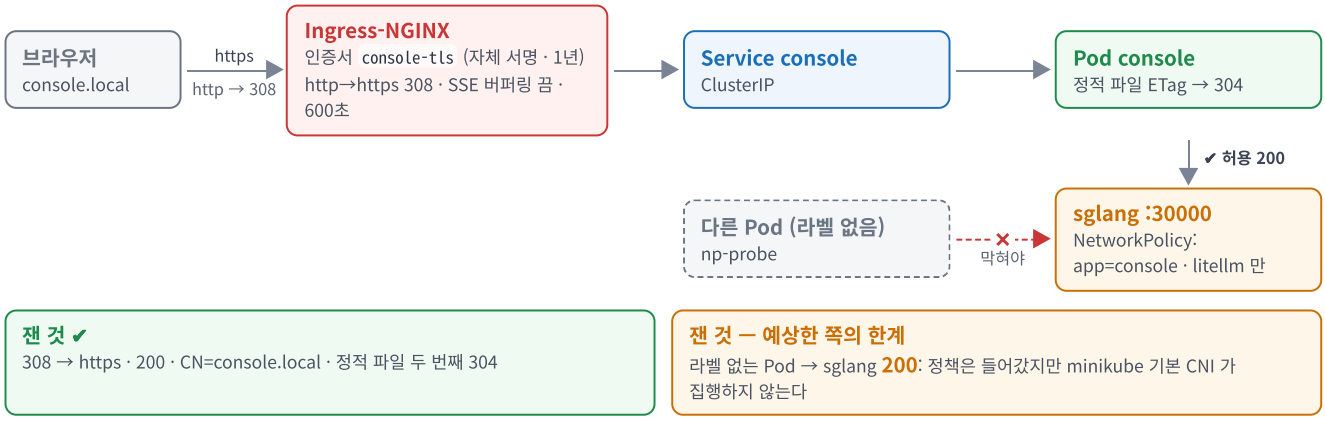
이미지는 미리 받아 두고(AMI · 캐시), 가중치는 가까운 저장소(로컬 NVMe · FSx)에서, 컴파일 결과 · CUDA 그래프는 캐시로 남기고, 최소 대수를 유지해 아예 0 에서 시작하지 않습니다.

08 엣지

https · 되돌림 · 방화벽 · CDN 흉내

완료 목표 상담 화면 앞에 Ingress-NGINX 를 두고 자체 서명 인증서로 https 를 켠다. http 는 https 로 되돌리고, 모델 서버는 상담 화면(과 09장의 게이트웨이)만 부를 수 있게 NetworkPolicy 를 건다. 정적 파일에 캐시 머리말을 붙여 두 번째 요청이 304 인지 본다.

그림 08 밖에서 들어오는 길은 하나



EKS 라면 ACM 인증서 + ALB(되돌림) · WAF · 보안 그룹, NetworkPolicy 는 VPC CNI 의 정책 기능이나 Calico/Cilium 이 집행. CDN 은 CloudFront — 여기선 Cache-Control 머리말만.

실행

Ubuntu Bash 08-001

```
minikube -p agent addons enable ingress
bash deploy/scripts/08-edge.sh
grep -q console.local /etc/hosts || echo "$(minikube -p agent ip) console.local" | sudo tee -a /etc/hosts
```

1. Ingress-NGINX 컨트롤러(ingress-nginx 네임스페이스)를 켠다 — 클러스터 밖에서 들어오는 http(s) 를 받아 Service 로 넘기는 문지기.
2. 인증서 만들기(`openssl` , `var/console.crt·key`) → Secret `console-tls` → `08-ingress.yaml` (Ingress + NetworkPolicy) → 측정 다섯 가지를 `results/cloud/08_edge.txt` 에. 부록 A8.
3. 브라우저로 보려면 `console.local` 이 minikube IP 를 가리켜야 한다. 그 뒤 `https://console.local/` (자체 서명이라 경고를 한 번 넘긴다).

관찰 ① http 는 `308` 과 `https://…` 로 되돌림 ② https 는 200, 인증서 `CN=console.local` ③ `/static/app.css` 두 번째 요청이 `304` (ETag) 이고 `Cache-Control: public, max-age=3600` ④ 라벨 없는 Pod 에서 모델 서버는 **막히나**(`000` , exit 28) — **minikube 기본 CNI 는 NetworkPolicy 를 강제하지 않는다.** 막히지 않으면(200) 그게 측정 결과다: 실제 클라우드의 CNI(Calico·Cilium, EKS 의 VPC CNI 정책)는 강제한다. 강제해 보려면 장애·복구. ⑤ 상담 화면 Pod 에서는 200.

장애·복구 NetworkPolicy 가 안 막히면 `minikube start -p agent --cni=calico` 로 **클러스터를 다시 만들어야** 한다(CNI 는 만들 때 정해진다) — 10GB 이미지와 PVC 를 잃으므로 이 워크북에선 하지 않고 적어 둔다. Ingress 가 `503` 이면 Service 이름·포트가 틀린 것, `404` 면 host 가 안 맞는 것(`--resolve` 또는 `/etc/hosts`).

정리 없음(09장이 이 Ingress 를 바꿔 쓴다).

성공 확인 아래 표.

항목	예상	결과 (2026-10-12, 노트북)
http → https 되돌림	308	**308** → `https://console.local/healthz`
https 상태 · 인증서	200 · CN=console.local, 365일	**200** · `CN=console.local` , 2026-10-12 ~ 2027-10-12
정적 파일 두 번째 요청	304 + Cache-Control	**304**(ETag). `Cache-Control` 은 안 보임 — 그 코드는 이미지 v2(10장)에 들어 있고 지금 Pod 는 v1. 10장 뒤 다시 본다
라벨 없는 Pod → 모델 서버	막힘(000) 또는 안 막힘(200, 기본 CNI)	**200 — 안 막힘.** 정책은 들어갔지만 기본 CNI 가 집행하지 않는다(예상한 쪽). 실제 클라우드와 다른 점으로 적는다
상담 화면 Pod → 모델 서버	200	200

첫 실행은 Ingress 컨트롤러의 검증 웹훅이 뜨기 전에 매니페스트를 넣어 `connection refused` — 스크립트가 컨트롤러를 기다린 뒤 넣게 고쳤다.

실제 클라우드에서는 (AWS)

이 워크북	AWS	다른 점
Ingress-NGINX + 자체 서명 인증서	**ALB** + **ACM** 인증서(자동 갱신) · Route 53	인증서를 사람이 갱신하지 않는다
`ssl-redirect` 308	ALB 리스너 규칙 80 → 443 리다이렉트	
NetworkPolicy (기본 CNI 는 미집행)	VPC CNI 의 NetworkPolicy · Calico · Cilium + **보안 그룹**	실제 클러스터는 집행한다
Cache-Control 머리말만	**CloudFront** + WAF	정적 파일은 엣지에서

연습 — 바꿔 보고 관찰하기

1. **되돌림 끄기.** `08-ingress.yaml` 의 `ssl-redirect` 를 `"false"` 로 → `curl -s -o /dev/null -w '%{http_code}' --resolve console.local:80:$(minikube -p agent ip) http://console.local/healthz` . 예측 → 200(평문으로 그대로). 되돌린다.
2. **다른 호스트 이름.** `--resolve other.local:443:<ip> https://other.local/healthz -k` . 예측 → 404 — Ingress 는 host 규칙으로 고른다.
3. **헤더 읽기.** `curl -skI --resolve … https://console.local/static/app.css` 로 `etag` · `cache-control` 을 보고, `If-None-Match` 를 붙인 요청과 비교.
4. **(선택) 집행되는 CNI.** `minikube start -p np --cni=calico --memory=3g` 로 작은 클러스터를 따로 만들어 같은 NetworkPolicy 를 넣으면 라벨 없는 Pod 가 막히는지(000) 본다. 끝나면 `minikube delete -p np` .

이해 확인

1. 이 구성에서 TLS 가 끝나는(복호화되는) 곳은? (가) 브라우저 (나) Ingress-NGINX (다) Service (라) Pod
2. NetworkPolicy 를 넣었는데 라벨 없는 Pod 가 모델 서버에 닿은 이유는?
3. 301 대신 308 로 되돌리는 차이는?
4. SSE 응답 앞에서 `proxy-buffering: off` 가 필요한 이유는?

면접에서 이렇게 묻는다

Q. 브라우저에서 Pod 까지 요청이 어떤 길로 가나요?

DNS 가 로드 밸런서 주소를 주고, 로드 밸런서(또는 Ingress)가 TLS 를 끝낸 뒤 host · 경로 규칙으로 Service 를 고르고, Service 가 라벨이 맞는 Ready Pod 중 하나로 보냅니다. 제 실습에선 http 는 308 로 https 로 되돌리고, 정적 파일은 ETag 로 두 번째부터 304 였습니다.

Q. 정책은 넣었는데 왜 안 막혔나요?

NetworkPolicy 는 선언일 뿐이고 집행은 CNI 가 합니다. minikube 기본 CNI 는 집행하지 않아서 라벨 없는 Pod 도 모델 서버에 200 으로 닿았습니다. 실제 클러스터에선 Calico · Cilium · VPC CNI 정책 기능이 집행하고, 그 위에 보안 그룹으로 한 겹 더 둡니다. '넣었다'와 '막혔다'를 따로 재야 하는 이유입니다.

09 관문

로그인 · 요청 제한 · LiteLLM 게이트웨이

완료 목표 Ingress 에 기본 인증(Basic Auth)과 초당 요청 제한을 더하고, 모델 서버 앞에 LiteLLM 게이트웨이를 두어 키 없는 요청을 401 로 막는다.

그림 09 사람은 로그인, 프로그램은 키



빠진 것 5주차의 CPU 대체 엔진(llama.cpp) — 노트북 메모리가 모델 서버 하나로 찼다. 대체 경로는 5주차 측정(요청 손실 0/60)으로 갈음.

5주차의 게이트웨이 구성을 그대로 Pod 로 옮긴다. 다만 5주차의 **CPU 대체 엔진(llama.cpp)은 넣지 않는다** — 노트북 메모리가 모델 서버 하나로 이미 찼다(07장). 대체 경로의 동작은 5주차 측정(요청 손실 0, 60개)으로 갈음하고 여기선 키 검사와 통과만 본다.

실행

Ubuntu Bash — 09-001

```
bash deploy/scripts/09-gateway.sh agent agent-pass
kubectl -n lab set env deploy/console ONPREM_MODEL=gateway ONPREM_MODEL_URL=http://litellm:4000/v1 && kubectl -n lab rollout status deploy/console
```

1 `openssl passwd -apr1` 로 비밀번호 해시 → Secret `basic-auth` → Ingress 를 `09-ingress-auth.yaml` (인증 + `limit-rps: 5`)로 교체 → LiteLLM(ConfigMap 설정, 키는 04장의 Secret `model-keys`) → 측정: 인증 없이/있이, 40 요청 동시(503 수), 게이트웨이 틀린 키/맞는 키/실제 답. 결과 `results/cloud/09_gateway.txt` . 부록 A9.

2 상담 화면이 모델 서버를 **게이트웨이 이름(`agent-llm-4b`)으로** 부르게 바꾼다(`defaults.MODELS["gateway"]` , 키는 `ONPREM_MODEL_KEY` 로 이미 들어 있다). 브라우저 `https://console.local/` 에 로그인 창이 뜨고, 관리자 화면의 모델이 `gateway` 다.

관찰 인증 없이 `401` , 계정으로 `200` . 40 요청 중 503 이 나온 수가 '초당 5 + 순간 5' 를 넘은 몫이다(네트워크 순서에 따라 25~35). 게이트웨이는 틀린 키 `401` , 맞는 키 `200` , 질문엔 `"content":"파란색"` 같은 답.

장애·복구 로그인 뒤 상담 화면의 SSE 가 끊기면 `proxy-buffering: off` 가 빠진 것. 503 이 0 이면 요청이 느려 초당 5 를 못 넘은 것 — `limit-rps` 를 2 로 내려 본다. LiteLLM 이 `CrashLoopBackOff` 면 ConfigMap 의 들여쓰기.

정리 없음. (요청 제한이 뒤 장의 측정을 방해하면 `kubectl apply -f deploy/manifests/08-ingress.yaml` 로 되돌린다 — 10장 이후 스크립트는 Ingress 를 거치지 않고 Service 로 직접 부르므로 영향 없다.)

항목	예상	결과 (2026-10-12, 노트북)
인증 없이 / 계정으로	401 / 200	**401 / 200**
40 요청 동시 → 503 수	25~35	**36**(통과 4) — 초당 5 + 순간 허용 5 보다 조금 더 엄격했다(40개가 1초 안에 들어가 '초당 5' 만 적용된 셈)
게이트웨이 틀린 키 / 맞는 키	401 / 200	**400 / 200** — LiteLLM 은 틀린 키를 400 으로 거부한다(401 이 아님, 거부는 같음)
게이트웨이 경유 답	한 단어 답	`"content":"파랑"` (model agent-llm-4b, 생각 모드 끔) · 상담 화면 `ONPREM_MODEL=gateway` 전환 완료

첫 실행은 `auth-realm` 에 한글을 넣어 검증기가 거부(`invalid value`) — ASCII 로 바꿨다. 스크립트가 거기서 멈춰 LiteLLM 이 안 떴고, 그 사이 게이트웨이로 바꾼 상담 화면은 `/readyz` (엔진에 닿아야 함)를 못 넘겨 기다렸다 — LiteLLM 이 뜨자 저절로 Ready.

실제 클라우드에서는 (AWS)

이 워크북	AWS	다른 점
Basic Auth (Ingress)	**Cognito** · IAM Identity Center(SSO) · ALB 인증 연동	사람 로그인은 회사 SSO 로
`limit-rps: 5` (IP 기준)	API Gateway 사용량 계획 · throttling, WAF rate-based rule	사용자 · API 키 단위로 제한
LiteLLM 게이트웨이 · 키	같은 자리에 LiteLLM · Kong · API Gateway, 키는 Secrets Manager	모델별 비용 · 사용량 집계
대체 엔진 없음(메모리)	다른 AZ · 다른 모델 · 관리형(Bedrock)으로 대체 경로	5주차: 1순위를 죽여도 손실 0/60

연습 — 바꿔 보고 관찰하기

1. **제한을 더 세게.** `limit-rps: "2"` 로 바꿔 40개 동시. 예측 → 503 이 더 늘어 38 근처.
2. **사용자 더하기.** `printf 'kim:%s\n' "$(openssl passwd -apr1 pw2)" >> var/auth` 뒤 Secret 재생성. 두 계정 모두 200, 틀린 비밀번호는 401.
3. **모델 이름 하나 더.** `09-litellm.yaml` 의 `model_list` 에 같은 엔진을 `agent-llm-4b-copy` 로 하나 더 → `kubectl apply` 후 `rollout restart` → `/v1/models` 에 둘이 보이는지.
4. **엔진이 없으면.** `kubectl -n lab scale deploy/sglang --replicas=0` 뒤 게이트웨이로 질문. 예측 → 몇 초 뒤 오류 — 대체 엔진이 없는 게이트웨이는 문지기일 뿐. 되돌린다.

이해 확인

1. 사람과 프로그램을 인증하는 방법이 다른 이유는?
2. IP 기준 요청 제한의 약점은? (힌트: 사내 NAT)
3. 모델 서버 앞에 게이트웨이를 두는 이유 세 가지는?
4. 40개 동시 요청 중 4개만 통과한 숫자는 설정값과 어떻게 이어지나?

면접에서 이렇게 묻는다

Q. LLM 게이트웨이는 왜 두나요?

한 이름으로 여러 엔진 · 모델을 부르고, 키로 누가 쓰는지 가르고, 사용량 · 비용을 모으고, 1순위가 죽으면 대체 엔진으로 넘기기 위해서입니다. 5주차에 1순위 SGLang 을 강제로 죽여도 CPU 대체 엔진으로 넘어가 요청 손실이 0(60개)이었고, 8주차엔 같은 게이트웨이를 Pod 로 올려 틀린 키를 거부하는 걸 확인했습니다.

Q. 요청 제한(rate limit)은 어떻게 설계하나요?

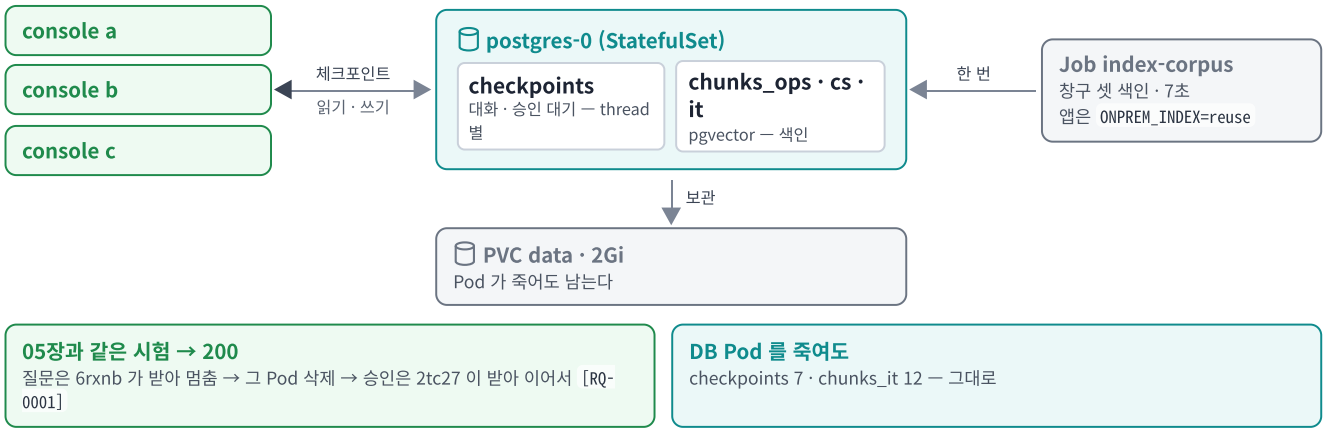
누구 기준(IP · 사용자 · 키)으로, 얼마나(초당 · 순간 허용), 넘으면 무엇을(429/503 + Retry-After) 정합니다. IP 기준은 사내 NAT 뒤 모두가 한 IP 라 함께 막히므로, 로그인 뒤엔 사용자 기준으로 바꿉니다. LLM 은 요청 수보다 토큰 수 기준이 맞습니다.

10 데이터

PostgreSQL + pgvector 하나로, 무상태 되찾기

완료 목표 05장에서 확인한 "승인 대기가 Pod 안에 있다"를 고친다. 대화·승인 대기(LangGraph 체크포인터)와 검색 색인(pgvector)을 PostgreSQL StatefulSet 하나에 두고, 색인은 Job 이 한 번 만들며, 상담 화면 3개 중 하나를 죽여도 승인이 이어지는지(200) 다시 잰다. DB Pod 를 죽여도 데이터가 남는지도 본다.

그림 10 상태는 전부 DB 에, Pod 는 비어 있다



EKS 라면 RDS for PostgreSQL + pgvector(또는 Aurora), 색인은 배치(Step Functions · Airflow), 대화 저장은 같은 RDS 나 DynamoDB.

2주차에 재 둔 pgvector 어댑터(`stores/pgvector.py`)와 7주차 웹의 체크포인터 자리를 그대로 쓴다. 바뀐 코드는 셋: ① 웹 `--store postgres`(`ONPREM_WEB_STORE`, `ONPREM_PG_DSN`) ② 창구 서버의 벡터 저장소를 `ONPREM_STORE` 로 고름(표 이름 `chunks_<창구>`) ③ `ONPREM_INDEX=reuse` 면 저장소에 이미 든 색인을 그대로 쓴다(문서 임베딩을 건너뛰어 앱 시작이 빠르다). 이미지에 `agent-pg` 묶음이 들어가므로 **이미지를 v2 로 다시 만든다**(03장 스크립트 그대로 — 코드 층만 바뀌어 1분).

실행

Ubuntu Bash 10-001

```
bash deploy/scripts/03-build.sh v2
bash deploy/scripts/10-db.sh
```

1 `agent-pg`(PostgreSQL 체크포인터·psycopg·pgvector)가 든 이미지 `onprem-agent:v2`. 설치 층이 바뀌므로 이번엔 캐시를 못 쓴다 — 1~2분.

2 PostgreSQL(StatefulSet + PVC 2Gi, 이미지 `pgvector/pgvector:pg16`) → 색인 Job(창구 셋, 해시 임베더) → 표 확인 → 상담 화면을 v2 + postgres 저장 + pgvector 재사용으로 바꾸고 3개로 → **05장의 Pod 죽이기 그대로** → DB 의 체크포인트 수 → **DB Pod 를 죽이고** 다시 세어 데이터가 남는지. 결과 `results/cloud/10_db.txt`. 부록 A10.

예상 05장과 같은 시험에서 이번엔 `HTTP 200` 과 `"done": "잠금 해제를 요청했습니다"` — 승인이 다른 Pod 에서 이어진다. DB Pod 를 지워도 `checkpoints`·`chunks_it` 행 수가 같다. 색인 Job 은 해시 임베더라 수 초.

관찰 — 임베더를 bge-m3 로 노트북 기본값(bge-m3)으로 색인하려면 Job 의 `ONPREM_EMBEDDER` 를 `{"kind":"sentence-transformers","model":"BAAI/bge-m3"}` 로 바꿔야 하는데 이미지에 torch 가 없다(03장 결정). 이 워크북에선 해시 임베더로 흐름만 보고, bge-m3 색인은 **호스트에서** 돌려 같은 DB 에 넣는 길을 적어 둔다: `kubectl -n lab port-forward svc/postgres 5432:5432 &` 뒤 `ONPREM_STORE='{"kind":"pgvector","dsn":"postgresql://agent:agent@localhost:5432/agent"}' ONPREM_EMBEDDER='{"kind":"sentence-transformers","model":"BAAI/bge-m3"}' python -c "from onprem_agent.agent.tool_server import build_search; build_search(['examples/desks/it/data'])"` . 그러면 Pod 쪽은 질문 임베딩만 하면 되는데 — 그것도 bge-m3 가 필요하다. 즉 **검색 임베더는 결국 앱 이미지에 들어가야** 하고(이미지 3GB), 그걸 피하려면 임베딩을 별도 서비스(임베딩 서버)로 뺀다. 이 결정은 9주차로 넘기고 여기 적는다.

장애·복구 색인 Job 이 `relation … does not exist` 면 DB 가 아직 안 떠서 — `rollout status sts/postgres` 뒤 Job 을 지우고 다시 (`kubectl -n lab delete job index-corpus`). 상담 화면이 `/readyz` 503 이면 `ONPREM_PG_DSN` 이 틀린 것(로그에 `connection refused`).

정리 상담 화면 1개로(`scale --replicas=1`). PostgreSQL 은 11·12장이 쓴다.

항목	예상	결과 (2026-10-12, 노트북)
색인 Job 시간 · chunks_it 행 수	수 초 · 수십	**7초** · 표 3개(chunks_ops·chunks_cs·chunks_it), IT **12행**. 첫 Job Pod 는 DB 의 DNS 가 생기기 전에 떠서 `failed to resolve host 'postgres'` 로 실패, 둘째가 성공(스크립트가 DB 준비 뒤 Job 을 만들게 고침)
승인 대기 중 Pod 삭제 → 같은 상담 번호 승인	**200** (05장은 409)	**200** — 질문은 `6rxnb` 가 받아 담당자 확인에서 멈춤 → 그 Pod 삭제 → 승인은 `2tc27` 이 받아 이어서 실행, `[RQ-0001] 계정 잠금 해제 요청 접수됨: E1063` . 체크포인트 7개 · 상담 1건이 DB 에
DB Pod 삭제 → 체크포인트·색인 행 수	같음	**7 / 12 그대로**(PVC)
이미지 v2 크기	v1 + 수십 MB	**214MB**(+6MB), 빌드 39초

답 글은 "요청이 처리되었습니다. [RQ-0001] … 하지만 최종 승인은 관리자에게 요청해야 합니다" — 실행은 됐는데 모델이 말끝을 흐렸다(거짓 보고의 반대 방향: 한 일을 덜 했다고 말함). 채점기는 성공으로 보지만 읽어 두는 종류의 답이다.

실제 클라우드에서는 (AWS)

이 워크북	AWS	다른 점
PostgreSQL StatefulSet + PVC	**RDS for PostgreSQL**(pgvector) 또는 Aurora — Multi-AZ · 자동 백업 · 스냅샷	DB 운영(백업 · 장애 조치 · 패치)을 맡긴다
체크포인터 = 같은 DB	같은 RDS, 또는 DynamoDB · ElastiCache	대화 상태는 작고 자주 쓴다
색인 Job (7초)	Step Functions · AWS Batch · Airflow(MWAA) 배치	자료가 바뀔 때마다 다시 색인
비밀번호 `agent`	Secrets Manager 자동 교체 + IAM DB 인증	

연습 — 바꿔 보고 관찰하기

1. **세 번 연속.** replicas 3 에서 `05-kill.sh` 를 세 번. 예측 → 3/3 `HTTP 200` .
2. **상담 번호를 DB 에서 찾기.** 상담 화면에서 질문 하나 → 화면의 상담 번호 → `kubectl -n lab exec sts/postgres -- psql -U agent -d agent -c "SELECT thread_id, count(*) FROM checkpoints GROUP BY 1;"` 에서 같은 번호.
3. **PVC 까지 지우면.** (되돌릴 각오로) `kubectl -n lab delete sts postgres && kubectl -n lab delete pvc data-postgres-0` 뒤 `10-db.sh` . 예측 → 체크포인트 0, 색인은 Job 이 다시 만든다. '저장소를 지우면 끝'을 눈으로.
4. **색인 재사용을 끄면.** `kubectl -n lab set env deploy/console ONPREM_INDEX-` → Ready 까지의 시간을 재사용일 때와 비교.

이해 확인

1. StatefulSet 이 Deployment 와 다른 점 두 가지는?

2. Pod 를 지울 때와 PVC 를 지울 때 데이터는 각각 어떻게 되나?

3. 05장의 409 가 10장에서 200 이 된 이유를 한 문장으로.

4. DB 를 클러스터 안(StatefulSet)에 둘 때와 RDS 에 둘 때의 장단점은?

면접에서 이렇게 묻는다

Q. 에이전트의 상태(대화 · 승인 대기)는 어디에 두나요?

앱 서버 밖의 공용 저장소입니다. LangGraph 체크포인터를 PostgreSQL 에 두자, 승인 대기 중에 그 Pod 를 죽여도 다른 Pod 가 같은 상담 번호로 이어서 실행했습니다(409 → 200). 같은 PostgreSQL 에 pgvector 로 검색 색인도 넣어 DB 하나로 끝냈고, DB Pod 를 죽여도 PVC 덕에 데이터가 남았습니다. 운영이라면 RDS 로 백업 · 장애 조치를 맡깁니다.

Q. 벡터 DB 는 무엇을 쓰나요?

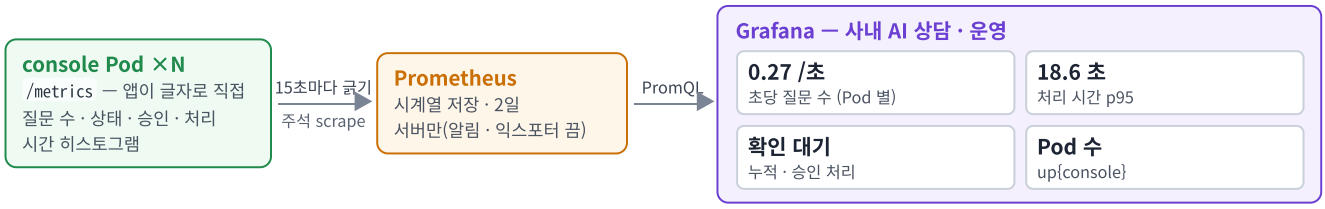
2~3주차에 같은 틀로 Chroma · Qdrant · Milvus · Elasticsearch · Weaviate · pgvector 를 비교했고, 이 규모에선 '이미 쓰는 PostgreSQL 에 확장 하나'인 pgvector 가 운영 부담이 가장 적었습니다. 수천만 건 이상 · 필터 검색이 많아지면 전용 DB 를 다시 잽니다.

11 관측

/metrics · Prometheus · Grafana

완료 목표 상담 화면이 숫자(질문 수 · 처리 시간 분포 · 확인 대기 · 승인 · 오류)를 `/metrics` 로 내놓고, Prometheus 가 15초마다 긁어 가며, Grafana 대시보드에 '초당 질문 수 · p95 처리 시간 · 확인 대기 수 · Pod 수'가 그려진다. 부하를 걸어 p95 가 숫자로 읽히는지 본다.

그림 11 숫자는 앱이 내놓고, 모으는 건 Prometheus, 그리는 건 Grafana



PromQL `histogram_quantile(0.95, sum by (le) (rate(agent_turn_seconds_bucket[5m])))` **빠진 것** Langfuse(에이전트 추적) — ClickHouse · Redis 가 더 필요. 대신 시험 결과 JSON 이 과업마다 호출을 남긴다. **EKS 라면** CloudWatch Container Insights · Amazon Managed Prometheus/Grafana.

지표는 **의존성 없이** 앱이 Prometheus 글자 형식으로 직접 쓴다(`web/app.py` 의 `/metrics`): `agent_chat_total` (질문 수), `agent_chat_status_total{status}` (답함·확인 대기·오류), `agent_approve_total` , `agent_turn_seconds` (히스토그램 — 0.5초부터 120초까지 9칸), `http_requests_total{path,code}` . 이걸 담은 이미지가 **v3** 다. Prometheus 는 `prometheus.io/scrape: "true"` 주석이 붙은 Pod 를 자동으로 긁는다 (04 매니페스트에 주석 추가).

실행

Ubuntu Bash 11-001

```
bash deploy/scripts/03-build.sh v3
bash deploy/scripts/11-observe.sh 60 4
kubectl -n lab port-forward svc/grafana 3000:80
```

1 `/metrics` 가 든 이미지 v3(코드 층만 바뀌어 1분).

2 helm 으로 Prometheus(서버만, 알림·익스포터 끔, 256Mi~768Mi)와 Grafana(128~384Mi, 데이터 원본과 대시보드를 파일로 심음) → 상담 화면에 스크레이프 주석 + v3 → Prometheus 대상 목록에 console 이 `up` 인지 → 부하 60초(동시 4, IT 창구 규정 질문) → 90초 뒤 Prometheus API 로 초당 질문 수 · p95 · 누적을 읽는다. 결과 `results/cloud/11_observe.txt` . 부록 A11. **지금 상담 화면은 게이트웨이 → 실제 모델**이라 질문 하나가 10~60초 걸리고 p95 도 그 크기다(규칙 모델이면 1초 아래). 어느 모델인지 스크립트가 같이 찍는다.

3 브라우저 `http://127.0.0.1:3000` (admin / lab-admin) → 대시보드 '사내 AI 상담 — 운영'. 부하가 끝난 뒤라 그래프의 꼬리만 보일 수 있다 — 2행을 한 번 더 돌리며 보면 움직인다.

관찰 Grafana 의 p95 선이 Prometheus API 의 숫자와 같다(같은 PromQL). '확인 대기'는 규정 질문 부하에서는 0 이고, 10장의 잠금 해제 상담을 한 번 하면 1 이 된다. **Langfuse(에이전트 추적)는 넣지 않았다** — v3 부터 ClickHouse·Redis·PostgreSQL 셋이 더 필요해 노트북 메모리(07장 예산) 밖이다. 대신 이 프로젝트의 추적은 이미 있다: 전체 시험의 `results/*.json` 이 과업마다 호출·안전 기능 발동을 남기고(7주차), 상담 화면의 '최근 상담'이 실시간 쪽이다. 실제 클라우드라면 Langfuse 를 별도 노드에 두고 LangGraph 콜백 하나로 붙인다 (6주차 조사).

장애·복구 대상 목록에 console 이 없으면 주석이 안 붙은 것(`kubectl -n lab get pod -l app=console -o jsonpath='{.items[0].metadata.annotations}'`). `up` 인데 지표가 없음 이면 아직 15초 주기가 안 지났거나 이미지가 v2(→ `/metrics` 404)다. Grafana 대시보드가 비어 있으면 ConfigMap 라벨이 아니라 `dashboardsConfigMaps` 값 이름(`lab`)이 맞는지.

정리 11장 서비스는 12장·13장이 쓰니 둔다. 내리려면 `helm -n lab uninstall prometheus grafana`.

항목	예상	결과 (2026-10-12, 노트북)
Prometheus 대상에 console (`up`)	1개 이상	**1, up**
부하 중 초당 질문 수 (Pod 별)	실제 모델: 0.1~0.3 · 규칙: 수십	**0.267**(게이트웨이 → Qwen3-4B(FP8), 생각 모드)
처리 시간 p95 (초)	실제 모델: 10~60 · 규칙: <1	**18.6초**
질문 누적 / 확인 대기 / 오류	n / 0 / 0	20 / (측정기 따옴표 실수로 못 읽음 — 고쳐 둠, 다음 실행에서)
Grafana 대시보드	네 칸 모두 숫자	(캡처)

이미지 v3 214MB(코드 층 +4KB), 빌드는 캐시. Prometheus·Grafana 설치 각 1분 안쪽(grafana 차트는 'deprecated' 경고 — 동작엔 문제 없음, 13장 `cloud-up` 에서도 같은 차트).

실제 클라우드에서는 (AWS)

이 워크북	AWS	다른 점
Prometheus 서버	**Amazon Managed Service for Prometheus**	저장 · 확장을 맡긴다
Grafana	**Amazon Managed Grafana** · CloudWatch 대시보드	SSO 로그인
로그는 `kubectl logs`	Fluent Bit → CloudWatch Logs · OpenSearch	Pod 가 사라져도 로그가 남는다
추적 없음(Langfuse 빠짐)	OpenTelemetry → X-Ray · Langfuse	요청 하나가 어디서 시간을 썼나
알림 없음	Alertmanager · CloudWatch 경보 → SNS · Slack	

연습 — 바꿔 보고 관찰하기

1. **PromQL 직접.** `kubectl -n lab port-forward svc/prometheus-server 9090:80` → 브라우저 `http://127.0.0.1:9090` → `rate(agent_chat_total[1m])` · `histogram_quantile(0.5, sum by (le) (rate(agent_turn_seconds_bucket[5m])))` (p50) 를 그려 본다.
2. **오류를 일부러.** 모델 서버를 0 으로 내리고 질문 두 번 → `agent_chat_status_total{status="오류"}` 가 2 오르는지. 되돌린다.
3. **알림 규칙 쓰기.** 'p95 가 30초를 5분 넘게 넘으면'을 PromQL 로 적어 본다: `histogram_quantile(0.95, …) > 30` + `for: 5m`. 부하로 넘겨 Alerts 화면에서 pending → firing 을 본다(알림 서버는 꺼져 있어 보내지는 않는다).

이해 확인

1. 질문 수(`agent_chat_total`)는 어떤 지표 형식인가? (가) gauge (나) counter (다) summary (라) histogram
2. counter 를 그대로 그리지 않고 `rate()` 를 씌우는 이유는?
3. 평균 대신 p95 를 보는 이유는?
4. 이 앱에서 가장 먼저 알림을 걸 숫자 두 개를 고르고 이유를 적어라.

면접에서 이렇게 묻는다

Q. 관측(observability)의 세 기둥은? LLM 앱에서는 무엇을 보나요?

지표 · 로그 · 추적입니다. LLM 앱에선 일반 지표(요청 수 · 오류율 · 지연 p95)에 더해 첫 글자 시간 · 초당 토큰 · 대기열 길이 · GPU 사용률, 그리고 업무 지표(승인 대기 · 근거 없는 답 · 거짓 실행 보고 같은 0 이어야 하는 칸)를 봅니다. 제 상담 화면은 의존성 없이 `/metrics` 를 직접 내놓고, 실제 모델 기준 p95 18.6초를 Grafana 에 그렸습니다.

Q. SLO 를 하나 정한다면?

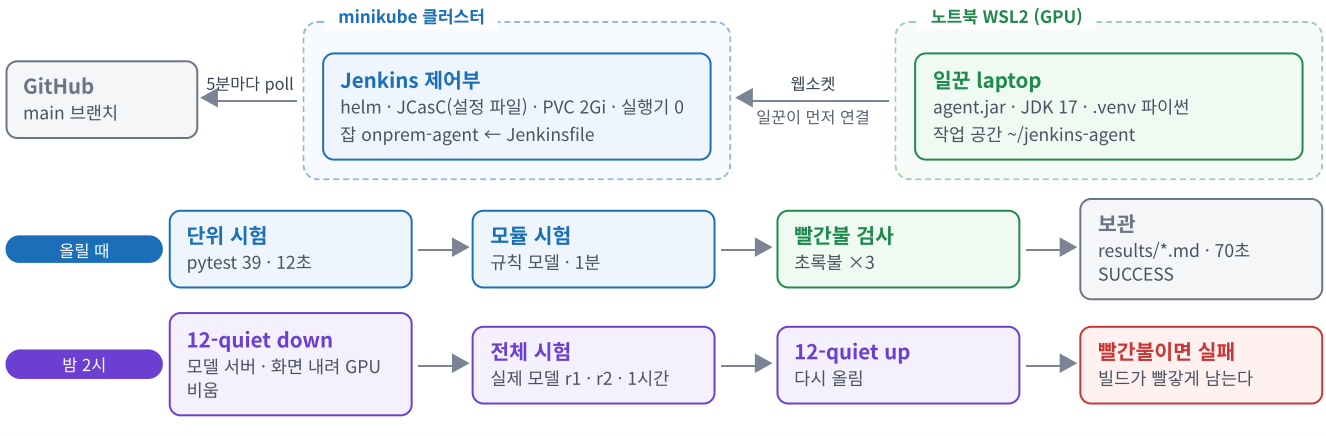
'질문의 95% 가 30초 안에 첫 단계(질문 이해)를 화면에 보인다' 처럼 사용자가 느끼는 것으로 정하고, 측정값(p95 18.6초)으로 여유를 확인한 뒤, 오류 예산을 넘으면 배포를 멈춥니다.

12 CI/CD

Jenkins 를 클러스터에, 노트북 GPU 를 일꾼으로

완료 목표 Jenkins 제어부를 helm 으로 클러스터에 올리고, **노트북을 인바운드 일꾼(`laptop` , 라벨 gpu)** 으로 붙인다. 저장소에 올리면(5분마다 poll) 단위·모듈 시험이 돌고, 매일 밤 2시에는 전체 시험(1시간, GPU)이 돈다 — 시작 전에 클러스터의 모델 서버·상담 화면을 내려 GPU 를 비우고 끝나면 올린다. 빨간불이면 빌드가 실패로 남는다.

그림 12 제어부는 안, 일꾼은 밖



핵심 모양 일꾼이 GPU 를 가진 쪽에 있다. **EKS 라면** GitHub Actions/CodeBuild + GPU 러너(셀프 호스트) · 배포는 ArgoCD(GitOps).

7주차의 시험 세 단계가 그대로 파이프라인이다(`Jenkinsfile`): `make test-unit` → `make test-module` → (타이머로 시작됐을 때만) `make test-full CONFIG=configs/agent_laptop_w7_split.yaml` 과 빨간불 검사. 일꾼은 노트북의 `.venv` 파이썬을 쓰되 코드는 Jenkins 가 내려받은 작업 공간의 것(`PYTHONPATH=$WORKSPACE/src`)이다 — 편집 설치가 `~/onprem-agent/src` 를 가리키므로 이렇게 덮어야 "올린 코드"를 시험한다.

실행

Ubuntu Bash 12-001

```
bash deploy/scripts/12-jenkins.sh
kubectl -n lab port-forward svc/jenkins 8081:8080
sudo apt install -y openjdk-17-jre-headless
bash deploy/scripts/12-jenkins.sh agent
```

1. helm 으로 Jenkins(JCasC 설정: 일꾼 `laptop` 정의, 저장소를 보는 파이프라인 잡 `onprem-agent` , 제어부 실행기 0). Ready 까지의 시간을 적는다(이미지 받기 포함 3~6분). 부록 A12.
2. **다른 창**에서 제어부를 노트북 8081 로 잇는다(켜 둔다). 브라우저 `http://127.0.0.1:8081` (admin / lab-admin).
3. 일꾼은 자바로 돈다 — 없으면 설치.
4. **또 다른 창**에서 일꾼을 붙인다: 제어부에서 `agent.jar` 와 `laptop` 의 비밀을 받아 웹소켓으로 연결(켜 둔다). Jenkins 화면의 '노드'에 `laptop` 이 온라인이 된다.

관찰 잡 `onprem-agent` 를 열어 **지금 빌드**를 누른다(또는 커밋을 하나 올리고 5분 기다린다) → 단위 → 모듈 두 단계가 노트북에서 돈다(모듈 시험 약 1분). 이 시간을 적는다. **밤 2시 전체 시험은 내일 아침** 빌드 기록과 `results/*_gate.md` (보관된 산출물)로 확인한다 — 12-quiet.sh 가 모델 서버를 내렸다 올린 로그도 거기 있다.

장애·복구 일꾼이 `offline` 이면 4행의 창이 닫혔거나 2행의 port-forward 가 끊긴 것(둘 다 켜 둬야 한다). 빌드가 `make: command not found` 면 일꾼 셸의 PATH — `.venv` 는 절대 경로라 괜찮고 `make` 는 WSL 에 있다. 밤 시험이 `CUDA out of memory` 면 12-quiet.sh 가 모델 서버 Pod 를 못 내린 것 — `kubectl -n lab get pods -l app=sglang` .

정리 두 창(port-forward · 일꾼)은 밤 시험을 보려면 켜 둔다. 끄면 밤 시험은 '일꾼 없음'으로 대기한다.

항목	예상	결과 (2026-10-12, 노트북)
Jenkins Ready 까지	3~6분	설정 오류 셋을 고치며 약 30분 — ① 플러그인 이름 `timestamps` → `timestamper` (init 반복 실패) ② JCasC `numExecutors` 가 차트 기본 설정과 충돌 → 차트 값으로 ③ 일꾼 `remoteFS` 가 없는 경로(`AccessDenied` , 빌드 #1 3.8초 실패) → 실제 폴더. 마지막 `helm upgrade` 뒤 82초에 `2/2 Running`
일꾼 `laptop` 온라인	됨	**됨** — 웹소켓 `Connected` , 응답 286ms. 빌드 중 9초 끊겼다 다시 붙음(`laptop is back online`)
올릴 때 빌드 (단위+모듈) 시간	1~2분	**#2 SUCCESS, 약 70초** — 단위 39 통과(12초), 모듈 5 통과 + 규칙 모델 평가 셋(기존 26 · 보호 20 · 경계 6 · 확인용 26 · IT 22/23), 빨간불 검사 초록불 ×3, 밤 단계는 `when` 으로 건너뜀, 산출물 보관
밤 전체 시험 (다음 날 확인)	1시간, 초록불	(2026-10-13 아침에)

로그의 `FileNotFoundError: ''` 는 Makefile 이 `CONFIG` 없이도 결과 이름을 계산하려던 것 — 조건부로 고침(빌드엔 영향 없었음).

실제 클라우드에서는 (AWS)

이 워크북	AWS / 업계	다른 점
Jenkins 제어부(클러스터)	GitHub Actions · CodePipeline/CodeBuild · GitLab CI	관리형 CI 가 흔하다
노트북 GPU 일꾼(인바운드)	셀프 호스트 GPU 러너(EC2 g5, 스팟) · 야간에만 켜기	GPU 시험은 비싸서 따로
`make test-*` 세 단계 + 빨간불	같은 구조 — PR 엔 빠른 시험, 밤엔 무거운 시험	
배포는 사람이 `kubectl apply`	**ArgoCD · Flux(GitOps)** — Git 이 원하는 상태	배포도 Git 리뷰를 거친다

연습 — 바꿔 보고 관찰하기

1. **실패를 보기.** Jenkins 화면의 빌드 #2 → **Replay** → Jenkinsfile 의 단위 시험 단계에 `sh 'exit 1'` 을 더해 실행. 예측 → 빨간 빌드, 뒤 단계는 건너뜀. 원본은 그대로다.
2. **일꾼이 없으면.** 일꾼 창(창 3)을 Ctrl+C 로 끄고 ▷. 예측 → 빌드가 대기열에서 `laptop is offline` 으로 기다린다. 다시 켜면 이어서 돈다.
3. **자동으로 도는지.** 아무 문서에 한 줄 고쳐 push → 5분 안에 빌드가 저절로(poll) 생기는지 본다.
4. **밤 시험 결과 읽기.** 다음 날 아침 빌드의 산출물에서 `*_gate.md` 를 열어 초록 · 노랑 · 빨강과 그 이유를 적는다.

이해 확인

1. 제어부의 실행기를 0 으로 둔 이유는?
2. 일꾼이 제어부로 먼저 연결하는(인바운드 · 웹소켓) 방식이 사내망에 유리한 이유는?
3. 전체 시험 전에 `12-quiet.sh down` 을 하는 이유는?
4. 모듈 시험이 규칙 모델(가짜 AI)을 쓰는데도 의미가 있는 이유는?

면접에서 이렇게 묻는다

Q. LLM 시스템의 CI 는 일반 앱과 무엇이 다른가요?

실제 모델은 느리고, 비싸고, 온도 0 에서도 흔들립니다. 그래서 세 단계로 나눴습니다 — 커밋마다 15초 단위 시험과 1분 모듈 시험(규칙 모델로 그래프 · 창구 서버 · 채점기를 끝까지, 기대 파일과 한 글자라도 다르면 실패), 밤에만 실제 모델로 같은 설정을 두 번 돌리는 전체 시험. 판정은 사람 대신 빨간불 검사가 합니다 — 위험 행동 · 개인정보 노출 · 거짓 실행 보고가 1건이라도 있으면 점수와 상관없이 실패입니다. Jenkins 일꾼을 GPU 가 있는 노트북에 두어 밤 시험이 GPU 를 쓰게 했습니다.

Q. GitOps 는 무엇인가요?

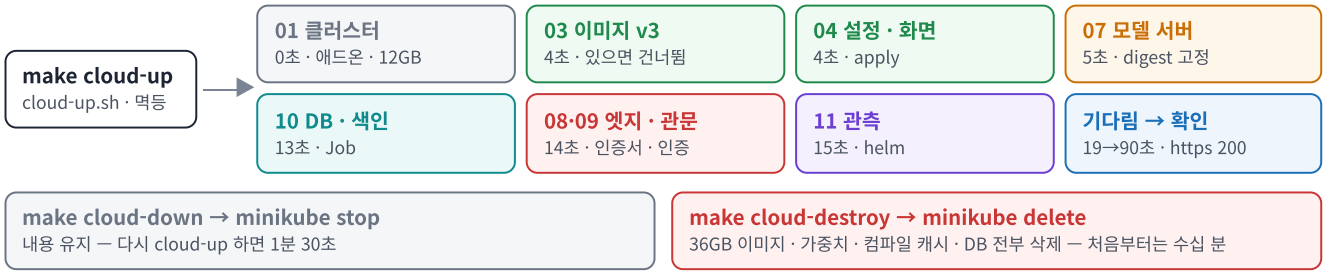
클러스터의 원하는 상태를 Git 에 두고, ArgoCD 같은 도구가 클러스터를 Git 에 맞춥니다. 배포가 PR 리뷰를 거치고, 되돌리기가 `git revert` 가 됩니다.

13 IaC

make cloud-up 한 줄, 처음부터 다시 올리는 시간

완료 목표 01~11장을 스크립트 하나(`deploy/scripts/cloud-up.sh`)로 순서대로, **다시 돌려도 되게(멱등)** 묶고, 클러스터를 지운 뒤 처음부터 올리는 데 걸리는 시간을 잰다. 이미지 태그 `latest` 는 digest 로 고정한다(C11 의 교훈).

그림 13 매니페스트·값 파일이 곧 설계도



멱등 '다시 안 함'이 아니라 '다시 해도 같은 결과'. kubectl apply · helm upgrade --install · --dry-run=client | apply. **실제라면** Terraform(클러스터·노드·RDS) + Helm 차트(앱) + ArgoCD.

실행

Ubuntu Bash 13-001

```
make cloud-up                          # 지금 상태에서 — 전부 이미 있으니 '확인'만 하고 몇 분 안에 끝나야 한다
make cloud-destroy && make cloud-up    # 처음부터 — 이미지 36GB 받기·가중치·컴파일 캐시가 다 사라지므로 길다. 시간을 적는다
```

1 멱등성 확인: 있는 것은 건너뛰고(`image ls` , `get job` , `apply` 의 unchanged) 끝에 `https 200` 과 Pod 목록. 걸린 시간이 '다시 돌리기 비용'이다.

2 **선택** — 사라지는 것: SGLang 이미지(36GB, 다시 받기), 가중치 PVC(4.4GB), JIT 캐시(07장 첫 시작의 긴 컴파일을 다시), DB. 인터넷과 컴파일 때문에 1시간을 넘길 수 있다. 할 때는 다른 일을 하면서. 끝나면 `results/cloud/13_cloud_up.txt` 의 `[초] 단계` 줄이 단계별 시간표다.

관찰 1행과 2행의 차이가 '캐시의 값'이다 — 이미지·가중치·컴파일 캐시 셋. 실제 클라우드에서 이 셋을 레지스트리·객체 저장소·베이스 이미지로 미리 두는 이유.

장애·복구 2행에서 07 단계가 길면 C11 의 첫 시작(컴파일)이다 — `kubectl -n lab logs -l app=sglang -f` 로 진행을 본다. Jenkins 는 기본 건너뜀(`WITH_JENKINS=1 make cloud-up` 으로 포함) — 일꾼은 사람이 붙여야 해서.

항목	예상	결과 (2026-10-12, 노트북)
1행: 있는 상태에서 `make cloud-up`	2~5분	**1분 30초** — 01~11 단계 19초(전부 있음·unchanged), 기다림 71초(모델 서버가 digest 고정으로 재시작 → 73초 Ready, 캐시 덕). 끝 `https 200` Pod `console-····-bqrpc` , Pod 10개 Running/Completed
2행: 처음부터 (선택)	40~90분	하지 않음 — 36GB 이미지와 컴파일 캐시(C11)를 잃는다. 하면 07 단계가 다시 수십 분
끝의 `https 200`	200	**200**

가중치·색인 Job 은 `ttlSecondsAfterFinished` 로 이미 지워져 있어 다시 만들어졌다(가중치는 캐시에 있어 수 초, 색인 7초) — 멱등이 '다시 안 함'이 아니라 '다시 해도 같은 결과'라는 뜻임을 보여 준다.

실제 클라우드에서는 (AWS)

이 워크북	AWS / 업계	다른 점
`cloud-up.sh` (kubectl · helm)	**Terraform**(VPC · EKS · 노드 그룹 · RDS · IAM) + Helm 차트 · Kustomize(앱) + ArgoCD	인프라와 앱의 경계를 나눈다
상태는 클러스터 자체	Terraform state 를 S3 + DynamoDB 잠금으로	여러 사람이 동시에 바꾸지 않게
환경 하나	dev · stage · prod 를 같은 코드 + 다른 변수로	

연습 — 바꿔 보고 관찰하기

1. **멱등 확인.** `make cloud-up` 을 두 번 연속. 예측 → 두 번째가 더 짧고, 결과(Pod 목록 · https 200)는 같다.
2. **드리프트.** `kubectl -n lab set env deploy/console ONPREM_MODEL=rule` 로 손으로 바꾼 뒤 `make cloud-up`. 예측 → `gateway` 로 되돌아간다 — 코드가 진실이다.
3. **(선택) 처음부터.** 다른 일을 하며 `make cloud-destroy && make cloud-up`, 단계별 시간표(`results/cloud/13_cloud_up.txt`)를 1행의 시간과 비교. 차이가 '캐시의 값'이다.

이해 확인

1. 멱등(idempotent)이란? (가) 한 번만 실행된다 (나) 여러 번 실행해도 결과가 같다 (다) 실패하면 되돌린다 (라) 병렬로 실행된다
2. 선언형(apply)과 명령형(create · scale)의 차이는?
3. 드리프트란 무엇이고 왜 위험한가?
4. 매니페스트에 이미지 digest 를 고정한 이유는?

면접에서 이렇게 묻는다

Q. 인프라를 코드로 관리하는 이유는?

재현 · 리뷰 · 되돌리기입니다. 제 실습 클러스터는 `make cloud-up` 한 줄로 올라가고, 다시 돌려도 같은 결과(1분 30초, https 200)가 나오며, 손으로 바꾼 것은 다음 실행에서 코드대로 되돌아갑니다. 실제라면 Terraform 으로 VPC · EKS · RDS 를, Helm · ArgoCD 로 앱을 관리하고 상태 파일은 S3 + DynamoDB 잠금에 둡니다.

Q. Terraform 과 Helm 의 경계는?

Terraform 은 클라우드 자원(네트워크 · 클러스터 · DB · IAM), Helm 은 클러스터 안의 앱입니다. 앱 배포를 Terraform 에 넣으면 앱이 바뀔 때마다 인프라 계획이 흔들려서 나눕니다.

마치며 이 워크북에서 잰 것

층	잰 것	결과
이미지	크기 · 빌드 · 캐시	208MB → 214MB(v2), 43초 → 1초(캐시)
로드 밸런서	Pod 별 요청 수	9·8·10 (측정기 결함 D45 를 잡은 뒤 30/30)
무상태	승인 대기 중 Pod 삭제	409(SQLite in Pod) → **200**(PostgreSQL 체크포인터)
HPA	1→4→1	지표 60초 지연, 늘기 즉시, 줄기 106초
GPU 모델 서버	첫 시작 · 둘째 시작 · 메모리	2시간 40분(C11) · **58초** · 7.9GB 안정 — 00장 예산 틀림
엣지	308/200/304 · NetworkPolicy	됨 · **기본 CNI 는 미집행**
관문	401/200 · 40 동시 · 게이트웨이	됨 · 36 차단 · 400/200, "파랑"
DB	DB Pod 삭제 뒤 데이터	7/12 유지
관측	p95 · 초당 질문	18.6초 · 0.27/s (실제 모델)
CI/CD	올릴 때 빌드	Jenkins + 노트북 일꾼, 70초 SUCCESS, 빨간불 검사 초록불 ×3 · 밤 시험은 다음 날
IaC	`make cloud-up` 다시 돌리기	1분 30초, https 200

세 가지가 감과 달랐다 — **메모리 예산**(모델 서버 4~5GB 가 아니라 7.9GB, 첫 시작은 12GB 넘게), **무상태는 저절로 되지 않는다**(Pod 를 늘릴 수 있어도 상태가 안에 있으면 늘린 게 아니다), **측정기부터 의심**(30회 중 3회 빠진 건 요청이 아니라 `kubectl run -i` 였다). 문제해결 이력 C9·C10·C11, D45.

부록 스크립트 한 줄씩

A0. deploy/scripts/install-minikube.sh `command -v X || {…}` : X 가 없을 때만 설치. kubectl 은 `dl.k8s.io` 의 최신 안정판, minikube 는 공식 릴리스, helm 은 공식 설치 스크립트. 끝에 세 버전을 찍는다.

A1. deploy/scripts/probe-gpu.sh `set -u` 만 켜고 `-e` 는 끈다 — 실패해도 끝까지 기록해야 판정이 되니까. 호스트 `nvidia-smi` 와 Docker 런타임 → minikube 상태 → 기다리기 세 가지(새 네임스페이스의 `default` 서비스 계정 · 장치 플러그인 DaemonSet 의 rollout · 노드의 `nvidia.com/gpu` 광고) → CUDA 기본 이미지 Pod 를 `--overrides` 로 `nvidia.com/gpu: 1` 을 요구하며 띄워 `nvidia-smi -L` → Pod 의 Events 와 노드 광고 수. 전부 `tee -a` 로 `results/cloud/01_gpu_probe.txt` 에 쌓인다(C9 의 교훈이 기다리기 세 줄).

A3. deploy/scripts/03-build.sh `docker build -q` (조용히, 이미지 id 만) 앞뒤로 `date +%s` 를 찍어 초를 센다 → `docker image inspect` 로 크기와 층 수 → `docker history` 로 층별 크기 → 호스트에서 `docker run -d --rm -p 18080:8080` 으로 띄워 30초 안에 `/healthz` 가 200 이 되는지 → 멈춤 → `minikube image load` 시간 → 노드 창고 목록. `tee` 로 `results/cloud/03_image.txt` .

A4. deploy/scripts/04-config.sh `kubectl create configmap --from-file=desk.yaml=<파일> --dry-run=client -o yaml | kubectl apply -f -` : '만들기'를 '있으면 덮기'로 바꾸는 상투구. Secret 도 같은 꼴, 값은 `${GATEWAY_KEY:-sk-local-dev}` (환경 변수가 있으면 그것, 없으면 개발용) — 키를 Git 에 두지 않는다.

A6. deploy/scripts/06-load.sh 부하 Pod 하나를 `kubectl run load … curl` 로 띄운다: 끝 시각까지 `동시 수` 만큼 `curl -X POST /api/chat … &` 를 띄우고 `wait` (한 묶음), 반복하며 요청 수를 센다. 호스트 쪽 루프는 (부하 초 + 90초)/10 번 돌며 `kubectl get hpa -o jsonpath` 로 현재 사용률/목표, `get deploy` 로 목표 Pod 수와 준비된 Pod 수를 한 줄씩 찍는다. 끝에 `top pod` 와 HPA 의 Events.

A8. deploy/scripts/08-edge.sh `openssl req -x509` 로 자체 서명 인증서(한 번만, `var/`) → `create secret tls` → Ingress·NetworkPolicy apply → 컨트롤러 준비 대기 → `curl --resolve console.local:443:<minikube ip>` 로 DNS 없이 host 를 맞춰 다섯 가지를 찍는다(308 · 200/인증서 · ETag 304 · 라벨 없는 Pod 의 curl · console Pod 의 urllib).

A9. deploy/scripts/09-gateway.sh `openssl passwd -apr1` 해시로 `var/auth` (htpasswd 꼴) → Secret `basic-auth` → 인증·제한 Ingress 로 교체 → LiteLLM apply → curl 401/200 → 40개 curl 을 `&` 로 동시에 띄워 코드를 `sort | uniq -c` → 1회용 Pod 에서 게이트웨이에 틀린 키·맞는 키·실제 질문.

A10. deploy/scripts/10-db.sh PostgreSQL apply·대기 → Job 완료 대기와 시간 → `psql \dt` ·행 수 → 상담 화면 `set image v2` + `set env` (postgres 저장·pgvector·reuse) → 3개로 → `05-kill.sh` 재사용 → `checkpoints` 집계 → `delete pod postgres-0` 뒤 다시 집계.

A11. deploy/scripts/11-observe.sh helm repo 추가 → 대시보드 ConfigMap → Prometheus·Grafana `upgrade --install --wait` → 상담 화면에 스크레이프 주석 `patch` (04 매니페스트를 `apply` 하면 10장의 env·이미지가 되돌아가므로) + 이미지 v3 → Prometheus `/api/v1/targets` 에서 console 대상 → 부하 Pod → `/api/v1/query` 로 PromQL 셋.

A12. deploy/scripts/12-jenkins.sh (기본) helm 으로 Jenkins 올리고 Ready 시간. (`agent`) `agent.jar` 를 제어부에서 받고, `/computer/laptop/jenkins-agent.jnlp` 에서 비밀(64자리 hex)을 뽑아 `java -jar agent.jar -webSocket` 으로 붙는다. `12-quiet.sh down|up` 은 sglang·console·litellm 을 0/1 로.

A13. deploy/scripts/cloud-up.sh 단계마다 `[경과초] 이름` 을 찍는다. 멱등 장치: `minikube status` 로 있으면 start 만, `image ls | grep` 으로 있으면 빌드 안 함, Job 은 `get job` 으로 있으면 다시 안 만듦, Secret 은 `--dry-run=client | apply` , helm 은 `upgrade --install` . 끝에 Ingress 를 통해 `https 200` 확인.

A5. deploy/scripts/05-lb.sh · 05-kill.sh 둘 다 `kubectl run … --image=curlimages/curl --rm -i --quiet` 로 클러스터 안에 1회용 curl Pod 를 띄운다. lb: `curl -w '%header{X-Served-By}'` 로 답한 Pod 이름만 N 줄 찍고 `sort | uniq -c` . kill: ① `/api/chat` 에 `-N` (버퍼 없이)으로 SSE 를 받다가 `approval` 사건이 보이면 끊고(`grep -m1`), `-D` 로 받은 머리말에서 답한 Pod 이름을 읽는다 ② 그 Pod 를 `delete --wait=false` ③ 두 번째 curl Pod 가 같은 상담 번호로 `/api/approve` 를 보내고 HTTP 코드와 답한 Pod 를 찍는다.

부록 이해 확인 정답과 해설

01 ① (나) — GPU 는 확장 자원이라 `limits` 로 정수 개를 요구한다. ② NVIDIA 장치 플러그인(DaemonSet) — 노드의 GPU 를 세서 kubelet 에 알린다. ③ 두 번째 Pod 는 `Pending` (`Insufficient nvidia.com/gpu`) — 기본은 나눠 쓰지 않는다. ④ 네임스페이스를 막 만들면 `default` 서비스 계정이 몇 초 뒤에 생기는데 그 전에 Pod 를 만들었다(C9).

02 ① (나) etcd. ② 스케줄러. ③ 다시 생기지 않는다 — 그 Pod 를 지켜 줄 컨트롤러(Deployment · ReplicaSet)가 없다. ④ 제어부(API 서버 · etcd 운영, 다중 AZ, 패치).

03 ① (나) — 층의 명령이나 복사하는 파일이 바뀌면 그 층부터 아래가 다시 만들어진다. ② 이미지가 3GB 를 넘어 올리기 · 되돌리기 · 노드 확장이 느려지고, 모델을 바꿀 때마다 이미지를 다시 만들어야 한다(코드와 재료를 떼어 둔다). ③ 노드에 이미지가 없으면 `ErrImageNeverPull` 로 뜨지 않는다. ④ 의존성 층이 코드 변경에 휘말리지 않게 — 코드만 바꾸면 설치 층은 캐시(43초 → 1초).

04 ① (나) base64 — 암호화가 아니다. ② 바뀌지 않는다 — subPath 마운트는 자동 갱신이 안 돼 재시작이 필요하다(디렉터리째 마운트하면 수십 초 뒤 갱신). ③ liveness 실패 → 컨테이너 재시작, readiness 실패 → Service 의 대상에서 뺀다(재시작 안 함). ④ 이미지를 다시 만들지 않고 창구 설정(화면 문구 · 업무 기능 이름 · 승인 카드)을 바꾼다 — 04-002 에서 확인.

05 ① (나) 라벨 셀렉터(`app: console`). ② port-forward 는 Pod 하나에 고정돼 Service 의 나눠 주기를 거치지 않는다. ③ 승인 대기가 질문을 받은 Pod 안(emptyDir 의 SQLite)에만 있어서, 승인 요청을 받은 다른 Pod 는 그 상담을 모른다. ④ 질문을 받은 그 Pod 가 죽으면 — sticky 는 같은 Pod 로 보낼 뿐 상태를 옮기지 않는다.

06 ① (다) requests.cpu — 그래서 requests 를 올리면 같은 부하에서 덜 늘어난다. ② metrics-server 가 첫 수치를 모으기 전이다(실측 약 60초). ③ 잠깐 조용하다고 바로 줄였다가 다시 늘리는 출렁임(플래핑)을 막으려고. ④ 병목이 CPU 가 아니라 GPU 라 CPU 사용률로는 부하가 안 보이고, 새 Pod 가 뜨는 데 58초~수십 분이라 늦게 늘리면 이미 늦다.

07 ① (나) — GPU 가 하나라 RollingUpdate 면 새 Pod 가 GPU 를 못 얻어 Pending 으로 멈춘다. ② PVC 에 남긴 컴파일 캐시(와 가중치) — 첫 시작의 FP8 커널 즉석 컴파일을 다시 하지 않는다. ③ 이 SGLang 의 `/health` 가 토큰 하나를 실제로 생성해 1초를 넘기는데 Probe 기본 timeout 이 1초였다. ④ 30초마다 `docker stats` 로 노드 메모리를 찍었다 — 안정 7.9GB, 첫 시작 12GB 넘게.

08 ① (나) Ingress-NGINX — 인증서 Secret `console-tls` 를 들고 있다. ② NetworkPolicy 는 선언이고 집행은 CNI 가 하는데 minikube 기본 CNI 는 집행하지 않는다. ③ 308 은 메서드와 본문을 그대로 유지하라는 뜻(POST 가 GET 으로 바뀌지 않음), 301 은 브라우저가 GET 으로 바꿀 수 있다. ④ 버퍼링하면 답이 다 만들어질 때까지 화면에 아무것도 안 온다 — SSE 는 사건이 생기는 대로 흘려야 한다.

09 ① 사람은 비밀번호 · SSO 로 '누구인가'를, 프로그램은 바꿔 끼우고 회수할 수 있는 키로 '어느 시스템인가'를 증명한다 — 수명과 회수 방식이 다르다. ② 사내 NAT 뒤 모든 사용자가 한 IP 로 보여 한 사람의 과부하가 모두를 막는다. ③ 한 이름으로 엔진 · 모델 고르기, 키 · 사용량 · 비용 관리, 장애 시 대체 경로. ④ 초당 5 에 순간 허용(5 × 배수 1)을 더해도, 40개가 1초 안에 몰려 처음 몇 개만 통과했다 — 측정은 4.

10 ① 고정 이름(postgres-0) · 순서 있는 시작 · Pod 마다 자기 PVC(volumeClaimTemplates). ② Pod 삭제 → 데이터 유지(PVC 가 남음), PVC 삭제 → 데이터 소멸. ③ 승인 대기가 Pod 안이 아니라 공용 DB 의 체크포인트에 있어 어느 Pod 가 받아도 이어갈 수 있다. ④ 클러스터 안: 싸고 한곳에서 관리, 대신 백업 · 장애 조치 · 업그레이드를 직접. RDS: 운영을 맡기지만 비용 · 네트워크 경계가 생긴다.

11 ① (나) counter — 늘기만 한다. ② counter 는 누적값이라 그래프가 계속 오를 뿐이고, `rate()` 로 '초당 얼마'를 봐야 변화가 보인다. ③ 평균은 느린 몇 건을 가린다 — 사용자 20명 중 1명이 겪는 지연을 보려면 p95. ④ 예: 처리 시간 p95(사용자 체감), 오류 수(모델 서버 다운) — 또는 '확인 대기'가 쌓이는 속도(담당자가 못 따라감).

12 ① 제어부에서 시험을 돌리면 클러스터 자원을 먹고 GPU 도 없다 — 일은 전부 GPU 가 있는 일꾼에서. ② 밖에서 노트북으로 들어오는 포트를 열 필요가 없다 — 일꾼이 나가는 연결만 한다. ③ 전체 시험이 GPU 와 메모리를 혼자 써야 한다(모델 서버 Pod 가 GPU 를 잡고 있으면 CUDA 메모리 부족). ④ 같은 질문에 늘 같은 답을 내서 '코드가 동작을 바꿨나'를 기대 파일로 정확히 잡는다 — 모델 품질이 아니라 배선을 시험한다.

13 ① (나). ② 선언형은 '이 모양이 되어라'(이미 그러면 아무것도 안 함), 명령형은 '이걸 해라'(두 번 하면 두 번 됨). ③ 실제 상태가 코드와 달라진 것 — 아무도 모르는 변경이 다음 배포에서 사라지거나 장애 원인이 된다. ④ `latest` 는 날마다 다른 이미지일 수 있다 — 07장에서 3주차와 다른 SGLang 이 받아져 설정이 어긋났다(C11).